

The logo consists of a large orange square on the left and a smaller orange square to its right, slightly lower. The word "NAVEX" is written in white, uppercase, sans-serif font inside the large orange square.

NAVEX™

NAVEX **IRM**™

NAVEX IRM Platform
API Reference Guide

This publication was written and produced at NAVEX Global, Inc., Overland Park, Kansas. This publication or any portion thereof is confidential and/or proprietary and may not be reproduced or used in any manner whatsoever without the express written permission of NAVEX Global, Inc.

© 2010 - 2022 NAVEX Global, Inc. All rights reserved. NAVEX®, NAVEX IRM™, the NAVEX IRM icon™, and Dynamic Content Framework™ are trademarks of NAVEX Global, Inc. and are registered in the United States. The trademarks and names of other companies and products mentioned herein are the property of their respective owners.

Last Update: February 17, 2022

Product Version: 6.1

Contents

About This Guide

INTRODUCTION

1: Fundamentals

NAVEX IRM API Services	9
Rules to NAVEX IRM API	11

SERVICES

2: Security Services API

Security Services API	15
Login	16
Ping	18
Logout	19
GetUser	20
GetUsers	24
GetUserCount	31
CreateUser	36
UpdateUser	48
DeleteUser	56
GetGroup	57
GetGroups	59
CreateGroup	63
UpdateGroup	66
DeleteGroup	70

3: Component Services API

GetComponent	72
--------------------	----

GetComponentList	74
GetComponentByAlias	76
GetFieldList	78
GetField	81
GetAvailableLookupRecords	83
GetRecord	86
GetRecords	88
GetRecordCount	92
GetDetailRecord	96
GetDetailRecords	99
GetLookupReportColumnFields	111
GetRecordAttachment	113
GetRecordAttachments	115
GetWorkflow	117
GetWorkflows	121
TransitionRecord	123
VoteRecord	125
CreateRecord	127
UpdateRecord	134
UpdateRecordAttachments	138
DeleteRecord	142
DeleteRecordAttachments	144
ImportFile	148

4: Report Services API

ExportReport	151
--------------------	-----

5: Assessment Services API

IssueAssessment	153
-----------------------	-----

APPENDICES

A: Language Identifiers

Language IDs	185
--------------------	-----

B: Troubleshooting Tips

Troubleshooting	188
Frequently Asked Questions	189

About This Guide

This guide provides the syntax for the various RESTful API verbs available with the NAVEX IRM API application. The intended audience for this reference guide is IT professionals and NAVEX IRM administrators who are responsible for importing and exporting data to and from the NAVEX IRM Platform.

The NAVEX IRM Platform API Reference Guide includes the following sections:

Introduction	Provides a list of the methods used for integrating external data with the NAVEX IRM Platform and the rules for using the NAVEX IRM API.
Security Services API	Provides a description of the functions for logging in and logging out of the NAVEX IRM Platform and methods for getting, creating, updating, and deleting users and groups.
Component Services API	Provides the available methods for retrieving, updating, and deleting data from the NAVEX IRM Platform.
Assessment Services API	Provides the available methods for initiating assessments into fields on DCF tables and on Master Detail records in the NAVEX IRM Platform.
Report Services API	Provides the available methods for reporting about the data in the NAVEX IRM Platform.
Appendices	Provides supporting information for language identifiers, troubleshooting tips, and frequently asked questions when using the NAVEX IRM API application.

Introduction

1: Fundamentals

This chapter provides a list of the methods for integrating external data with the NAVEX IRM Platform and the rules for using the NAVEX IRM API.

NAVEX IRM API Services	9
Rules to NAVEX IRM API	11

NAVEX IRM API Services

The NAVEX IRM Application Programming Interface (API) provides a set of Representational State Transfer (REST) APIs that allows access and integration to custom user-created data components stored in the NAVEX IRM Platform. The calls you use to access the NAVEX IRM API resources are grouped by Security Services, Component Services, Report Services, and Assessment Services.

Security Services API	
Call	Description
CreateGroup	Creates a group.
CreateUser	Creates a user account.
DeleteGroup	Deletes a group.
DeleteUser	Deletes a user account.
GetGroup	Returns all supporting fields of a given group.
GetGroups	Returns the ID and Name for groups.
GetUser	Returns all fields for a given user.
GetUserCount	Returns an integer representing the count of all Full Users, Vendors Contact Users, and Awareness Users.
GetUsers	Returns a list of users and supporting fields.
Login	Accepts an username and password, verifies them, and provides an encrypted cookie that can be used to authenticate additional API transactions.
Logout	Terminates a session.
Ping	Refreshes a valid session.
UpdateGroup	Updates a group.
UpdateUser	Updates a user account.

Component Services API	
Call	Description
CreateRecord	Creates a new record within the specified component of the application.
DeleteRecord	Deletes a selected record from within a chosen component.
DeleteRecordAttachments	Deletes the specified attachments from the provided document fields on a specific record.
GetAvailableLookupRecords	Retrieves records that are available for population for a lookup field.
GetComponent	Retrieves a component specified by its ID.
GetComponentByAlias	Retrieves a component specified by its Alias.

Component Services API	
Call	Description
GetComponentList	Returns a complete list of all components available to the user based on account permissions.
GetDetailRecord	Retrieves record information based on the provided component ID and record ID, with lookup field report details.
GetDetailRecords	Retrieves details for each record specified.
GetField	Retrieves details for a field within a component.
GetFieldList	Retrieves a detailed field list for a component specified by its ID.
GetLookupReportColumnFields	Retrieves the field information of each field in a field path that corresponds to a lookup report column.
GetRecord	Returns the set of fields for a given record for which you have permissions within a component.
GetRecordAttachment	Retrieves a single attachment for a given field on a record.
GetRecordAttachments	Retrieves a list of information for all attachments associated with the provided component ID, record ID, and field ID.
GetRecordCount	Returns the number of records in a given component.
GetRecords	Returns an array of records within a chosen component.
GetWorkflow	Retrieves workflow details and all workflow stages specified by ID.
GetWorkflows	Retrieves all workflows for a component specified by its alias.
ImportFile	Queues a job to import a file for a defined import template.
TransitionRecord	Transitions a record in a workflow stage.
UpdateRecord	Updates a record within the specified component of the application.
UpdateRecordAttachments	Adds new attachments and/or updates existing attachments to the provided documents field(s) on a specific record, where the FileData is represented as a Base64 string.
VoteRecord	Casts a vote for a record in a workflow stage.

Report Services API	
Call	Description
ExportReport	Outputs report data to a CSV, PDF, or XLSL file format.

Assessment Services API	
Call	Description
IssueAssessment	Issues an assessment according to a specified schedule.

Rules to NAVEX IRM API

The NAVEX IRM API resources are referenced by the URL path with standard HTTP verbs to access the different methods. Requests are made using the instance name of the NAVEX IRM installation, the port of the individual site followed by the applicable service and the available call associated with the service.

URL: `https://[instance name]:[port]/[service]/[call]`

Methods

The methods you use to access the NAVEX IRM API resources are GET, POST, and DELETE.

Method	Description
GET	Retrieves information about a resource.
POST	Creates a resource.
DELETE	Deletes a resource or related component of a resource.

Access Configuration

The methods you use to access the NAVEX IRM API resources are grouped by Security Services APIs, Component Services APIs, Report Services APIs, and Assessment Services APIs. API methods are case sensitive. Verify that you are entering the proper syntax and case for the given call as documented.

Type	API Resources Group
All logins	SecurityService
All data gathering/manipulation requests	ComponentService
All reporting requests	ReportService
All assessment requests	AssessmentService

Permissions

Permissions for the API are determined by the logon credentials used to access it. API access uses the permission settings for an account configured within the Security Roles section of the NAVEX IRM Platform.

The credentials are accessed using the Login method in the SecurityService API. The credentials return a cookie that is then passed as authorization to all of the other portions of the API and is detailed in the SecurityService API.

cURL

While the REST Framework supports almost any development language, examples in this documentation are provided in cURL and can be executed in a standalone command prompt. cURL is a tool to transfer data from or to a server, using HTTP(s) to connect to the web service API, and return an XML or JSON response that can be stored and parsed.

Some basic cURL command switches that may be helpful are shown below. For a list of curl commands and options, type curl --help at a command prompt.

Switches		
Switch	Option	Use
-b	--cookie	Passes the data to the server in the cookie header. This option provides authentication.
-c	--cookie-jar	Specifies to which file you want curl to write all cookies after a completed operation.
-H	--header	Passes custom header(s) to the server; includes header information, such as Content-Type.
-d	--data <data>	Sends specified data in a POST request to the server; uses @filename.xml to read the input data.

XML and JSON

In addition to cURL examples, XML and JSON samples illustrate how a method may be utilized in your development environment. In some samples, due to size constraints, an ellipse (...) indicates an incomplete list of items returned.

Data Structure

A key component of the NAVEX IRM architecture is the Dynamic Content Framework (DCF) which allows you to build customized, dynamic tables to store a variety of data elements. The DCF also allows for complex relational interaction between custom and permanent data elements. As such, the content is completely customized to each business and does not have a fixed structure.

Some terms will be useful to understand when using the NAVEX IRM API to access custom content.

Term	Description
Component	A single table in the NAVEX IRM Platform.
Field	A definition for a single piece of information in a component, for example, the address of a building. Each field is restricted to a user-defined data type.
Record	A complete grouping of multiple fields for a single identifier. For example, a record may consist of a name, address, and phone number for an employee.

Services

2: Security Services API

This chapter includes a description of the functions for logging in and logging out of the NAVEX IRM Platform and methods for getting, creating, updating, and deleting users and groups.

Security Services API	15
Login	16
Ping	18
Logout	19
GetUser	20
GetUsers	24
GetUserCount	31
CreateUser	36
UpdateUser	48
DeleteUser	56
GetGroup	57
GetGroups	59
CreateGroup	63
UpdateGroup	66
DeleteGroup	70

Security Services API

The Security Services API provides the available methods for logging in and logging out of the NAVEX IRM Platform. The Login call generates an encrypted cookie in the return header. This cookie must be captured and used as a parameter in all ComponentServices, ReportServices, and AssessmentServices calls to verify identity and set permissions.

Permissions for the data that the API can access are based on permissions of the Logon account that is used. For example, if the login account has read-only permissions to an asset table, the API returns data for user viewing but does not allow update or delete functions to be performed.

NOTE: API methods are case-sensitive. Verify that you are entering the proper syntax and case for the Security Services call as documented.

Login

Accepts an username and password, verifies it, and provides an encrypted cookie that can be used to authenticate additional API transactions.

Permissions	The authentication account that logs into the application must have:		
	• API Access enabled in the NAVEX IRM Platform		
URL:	https://[instance-name]:[port]/SecurityService/Login		
Method:	POST		
Input:	Username (string):	Username for the NAVEX IRM application account.	
	Password (string):	Password for the NAVEX IRM application account.	
Output:	A cookie to establish sessions within the component services API.		

Examples

In this example, cURL uses the `-c` option to store the encrypted cookie that is returned in a file. The cookie can be sent to authenticate all data manipulation commands in the ComponentService API with a single cURL command line switch (`-b`). The response sample includes both the header of the response (to view the cookie) and the XML response.

XML REQUEST (cURL)

```
curl -c cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d
"<Login><username>username</username><password>password</password></Login>"
https://api.lockpath.app:4443/SecurityService/Login
```

XML RESPONSE

```
<boolean xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</boolean>
```

JSON REQUEST (cURL)

```
curl -c cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d "{ \"username\": \"user1\", \"password\": \"12345\" }"
https://api.lockpath.app:4443/SecurityService/Login
```

JSON RESPONSE

```
true
```

Auto-provision an account using LDAP

If Auto-Provision is enabled for LDAP Profile and the authenticated user does not have a NAVEX IRM application account, one will be created with API access enabled and then authenticated to the application.

Input: ldapSettingsId (integer): LDAP Profile ID within application.

Examples

XML REQUEST (cURL)

```
curl -c cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d
"<Login><username>username</username><password>password</password><ldapSettingsId>1</ldap
SettingsId></Login>" https://api.lockpath.app:4443/SecurityService/Login
```

XML RESPONSE

```
<boolean xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</boolean>
```

JSON REQUEST (cURL)

```
curl -c cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d "{ \"username\": \"user1\", \"password\": \"12345\", \"ldapSettingsId\": \"1\"}"
https://api.lockpath.app:4443/SecurityService/Login
```

JSON RESPONSE

```
true
```

Ping

Refreshes a valid session.

Permissions: The authentication account must have:

- API Access enabled in the NAVEX IRM Platform

URL: `https://[instance-name]:[port]/SecurityService/Ping`

Method: GET

Input: No input allowed.

Examples

Ping uses the default GET method, so the method does not need to be specified in the request.

XML REQUEST (cURL)

```
curl -b cookie.txt https://api.lockpath.app:4443/SecurityService/Ping
```

XML RESPONSE

```
<boolean xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</boolean>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/SecurityService/Ping
```

JSON RESPONSE

```
true
```

Logout

Terminates a session.

Permissions: The authentication account that logs into the application must have:

- API Access enabled in the NAVEX IRM Platform

URL: `https://[instance-name]:[port]/SecurityService/Logout`

Method: GET

Input: No input allowed.

Examples

In this example, cURL uses the -b option to retrieve a stored session, authenticate it, and then terminate the session.

XML/JSON REQUEST (cURL)

```
curl -b cookie.txt https://api.lockpath.app:4443/SecurityService/Logout
```

XML/JSON RESPONSE

```
true
```

GetUser

Returns all fields for a given user.

Permissions: The authentication account must have:

- Administrative Access - Read permissions to Administer - Users

Vendor Contact accounts must also have:

- General Access - Read permissions to Vendor Profiles
- View permissions to Vendor Profiles workflow stage
- Vendor Profiles record permissions

URL: `https://[instance name]:[port]/SecurityService/GetUser?id={USERID}`

Method: GET

Input: Id (integer): The Id of the desired user.

Language (integer): The User object of the GetUser call reveals the language in use in the NAVEX IRM Platform. The Language object works in combination with the Preferred Locale feature. When one of the languages with a corresponding locale code is active in the NAVEX IRM Platform, the Preferred Locale field value in My Profile preferences is set for the user. In the NAVEX IRM Platform, the default language is English, and since English has a related locale code, the default language value is "1033."

If an API request returns a language that is not available, or if a language is not active in the instance, the error message "Invalid Language ID" returns.

Languages

The preferred language of the user is stored in the user profile and is revealed in Setup > Platform > Multilingual > Language. You can hover the cursor over the language name in the list to obtain the ID.

The following list includes the language ID, language name, locale ID, and locale name for the languages with corresponding locales. For a list of all languages and language IDs available in the NAVEX IRM Platform, see [Language IDs](#) in this appendix.

Language ID	Language Name	Locale ID	Locale Name
9	English	1033	English (United States)
9	English	2057	English (United Kingdom)
12	French	1036	French (France)
16	Italian	1040	Italian (Italy)
10	Spanish	2058	Spanish (Mexico)
10	Spanish	3082	Spanish (Spain)

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"  
https://api.lockpath.app:4443/SecurityService/GetUser?id=1055
```

XML RESPONSE

```
<UserItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">  
  <Id>1055</Id>  
  <FullName>User, Test</FullName>  
  <Username>testuser</Username>  
  <Active>true</Active>  
  <Locked>false</Locked>  
  <Deleted>false</Deleted>  
  <AccountType>1</AccountType>  
  <FirstName>Test</FirstName>  
  <MiddleName/>  
  <LastName>User</LastName>  
  <Title/>  
  <Language>1033</Language>  
  <EmailAddress>test@user.com</EmailAddress>  
  <HomePhone/>  
  <WorkPhone/>  
  <MobilePhone/>  
  <Fax/>  
  <Manager>  
    <Id>12</Id>  
    <FullName>Name, User</FullName>  
  </Manager>  
  <Department>  
    <Id>10</Id>  
    <Name>Anonymous Incident Analysts</Name>  
  </Department>  
  <IsSAML>false</IsSAML>
```

```

<IsLDAP>false</IsLDAP>

<SecurityConfiguration>
    <Id>1</Id>
    <DisplayName>Administrator Configuration</DisplayName>
</SecurityConfiguration>

<APIAccess>false</APIAccess>

<Groups>
    <Group>
        <Id>13</Id>
        <Name>Retail Stores</Name>
    </Group>
</Groups>

<SecurityRoles>
    <SecurityRole>
        <Id>1</Id>
        <Name>Administration</Name>
    </SecurityRole>
</SecurityRoles>

<FunctionalRoles/>
</UserItem>

```

JSON REQUEST (cURL)

```

curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json"
https://api.lockpath.app:4443/SecurityService/GetUser?id=1055

```

JSON RESPONSE

```

{
    "Id": 1055,
    "FullName": "User, Test",
    "Username": "testuser",
    "Active": true,
    "Locked": false,
    "Deleted": false,
    "AccountType": 1,
    "FirstName": "Test",

```

```

    "MiddleName": "",
    "LastName": "User",
    "Title": "",
    "Language": 1033,
    "EmailAddress": "test@user.com",
    "HomePhone": "",
    "WorkPhone": "",
    "MobilePhone": "",
    "Fax": "",
    "IsSAML": false,
    "IsLDAP": false,
    "SecurityConfiguration": {
        "Id": 1,
        "DisplayName": "Standard Configuration"
    },
    "APIAccess": false,
    "Groups": [],
    "SecurityRoles": [
        {
            "Id": 1,
            "Name": "End User"
        }
    ],
    "FunctionalRoles": []
}

```

GetUsers

Returns a list of users and supporting fields. You can filter the list to return only certain user types, such as Active, Deleted, or users by a particular AccountType or Vendor Contacts by a certain Vendor Profile.

Permissions: The authentication account must have:

- Administrative Access - Read permissions to Administer - Users

Vendor Contact accounts must also have:

- General Access - Read permissions to Vendor Profiles
- View permissions to Vendor Profiles workflow stage
- Vendor Profiles record permissions

URL: `https://[instance name]:[port]/SecurityService/GetUsers`

Method: POST

Input: `pageIndex` (integer): The index of the page of result to return. Must be ≥ 0 .
`pageSize` (integer): The size of the page results to return. Must be ≥ 1 .
`FieldFilter` (optional): The filter parameters the users must meet to be included.

Field Filters

Use filters to return only the users meeting the selected criteria.

ShortName	FilterType	Value
Active	5 (Equal To)	True, False
	6 (NotEqualTo)	
Deleted	5 (Equal To)	True, False
	6 (NotEqualTo)	
AccountType	5 (Equal To)	1 (Full User), 2 (Vendor, VendorContact), 4 (Awareness User)
	6 (NotEqualTo)	
	10002 (Contains Any)	
Vendor	5 (Equal To)	Vendor Profile ID for the Vendor Contact

Field Filter (Active) Example

XML	JSON
<pre> <FieldFilter> <Field> <ShortName>Active</ShortName> </Field> <FilterType>5</FilterType> <Value>False</Value> </FieldFilter> </pre>	<pre> "filters": [{ "Field": { "ShortName": "Active" }, "FilterType": "5", "Value": "false" }] </pre>

Field Filter (Deleted) Example

XML	JSON
<pre> <FieldFilter> <Field> <ShortName>Deleted</ShortName> </Field> <FilterType>6</FilterType> <Value>True</Value> </FieldFilter> </pre>	<pre> "filters": [{ "Field": { "ShortName": "Deleted" }, "FilterType": "6", "Value": "true" }] </pre>

Field Filter (AccountType) Example

XML	JSON
<pre> <FieldFilter> <Field> <ShortName>AccountType</ShortName> </Field> <FilterType>10002</FilterType> <Value>1 2</Value> </FieldFilter> </pre>	<pre> "filters": [{ "Field": { "ShortName": "AccountType" }, "FilterType": "10002", "Value": "1 2" }] </pre>

Field Filter (Vendor) Example

XML	JSON
<pre> <GetUsers> <pageIndex>0</pageIndex> <pageSize>100</pageSize> <filters> <FieldFilter> <Field> <ShortName>Vendor</ShortName> </Field> <FilterType>5</FilterType> <Value>7</Value> </FieldFilter> </filters> </GetUsers> </pre>	<pre> { "pageIndex": "0", "pageSize": "100", "filters": [{ "Field": { "ShortName": "Vendor" }, "FilterType": "5", "Value": "7" }] } </pre>

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @GetUsersInput.xml
```

```
"https://api.lockpath.app:4443/SecurityService/GetUsers"
```

XML REQUEST (GetUsersInput.xml)

```
<GetUsers>
  <pageIndex>0</pageIndex>
  <pageSize>4</pageSize>
  <filters>
    <FieldFilter>
      <Field>
        <ShortName>Active</ShortName>
      </Field>
      <FilterType>5</FilterType>
      <Value>true</Value>
    </FieldFilter>
  </filters>
</GetUsers>
```

XML RESPONSE

```
<UserList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <User>
    <Id>9</Id>
    <FullName>User, Test</FullName>
    <Username>testuser</Username>
    <Active>true</Active>
    <Deleted>false</Deleted>
    <AccountType>1</AccountType>
  </User>
  <User>
    <Id>10</Id>
    <FullName>User, VendorContact</FullName>
    <Username>vendorcontactuser</Username>
```

```

        <Active>true</Active>

        <Deleted>>false</Deleted>

        <AccountType>2</AccountType>

        <Vendor>

            <Id>5</Id>

            <DisplayName>Vendor</DisplayName>

        </Vendor>

    </User>

    <User>

        <Id>11</Id>

        <FullName>User, API</FullName>

        <Username>APIuser</Username>

        <Active>true</Active>

        <Deleted>>false</Deleted>

        <AccountType>1</AccountType>

    </User>

    <User>

        <Id>12</Id>

        <FullName>User, Test2</FullName>

        <Username>test2user</Username>

        <Active>true</Active>

        <Deleted>>false</Deleted>

        <AccountType>1</AccountType>

    </User>

</UserList>

```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @GetUsersInput.json
```

```
"https://api.lockpath.app:4443/SecurityService/GetUsers"
```

JSON REQUEST (GetUsersInput.json)

```

{

    "pageIndex": "0",

```

```

    "pageSize": "4",
    "filters": [
      {
        "Field": {
          "ShortName": "Active"
        },
        "FilterType": "5",
        "Value": "true"
      }
    ]
  }
}

```

JSON RESPONSE

```

[
  {
    "Id": 9,
    "FullName": "User, Test",
    "Username": "testuser",
    "Active": true,
    "Deleted": false,
    "AccountType": 1
  },
  {
    "Id": 10,
    "FullName": "User, VendorContact",
    "Username": "vendorcontactuser",
    "Active": true,
    "Deleted": false,
    "AccountType": 2,
    "Vendor": {
      "Id": 5,
      "DisplayName": "Vendor"
    }
  }
]

```

```
    },  
    {  
      "Id": 11,  
      "FullName": "User, API",  
      "Username": "APIuser",  
      "Active": true,  
      "Deleted": false,  
      "AccountType": 1  
    },  
    {  
      "Id": 12,  
      "FullName": "User, Test2",  
      "Username": "test2user",  
      "Active": true,  
      "Deleted": false,  
      "AccountType": 1  
    }  
  ]  
}
```

GetUserCount

Returns an integer representing the count of all Full Users, Vendors Contact Users, and Awareness Users in the application. Using the optional FieldFilter you can return records for only certain user types, such as Active, Deleted, or users by a particular AccountType or Vendor Contacts by a certain Vendor Profile.

Permissions: The authentication account must have:

- Administrative Access - Read permissions to Administer - Users

Vendor Contact accounts must also have:

- General Access - Read permissions to Vendor Profiles
- View permissions to Vendor Profiles workflow stage
- Vendor Profiles record permissions

URL: `https://[instance name]:[port]/SecurityService/GetUserCount`

Method: POST

Input: FieldFilter (various) (optional): The filter parameters the users must meet to be included.

Field Filters

Use filters to return the user count meeting the selected criteria.

FieldFilter (ShortName)	FilterType	Value
Active	5 (Equal To)	True, False
	6 (NotEqualTo)	
Deleted	5 (Equal To)	True, False
	6 (NotEqualTo)	
AccountType	5 (Equal To)	1 (Full User), 2 (Vendor Contact), 4 (Awareness User)
	6 (NotEqualTo)	
	10002 (Contains Any)	
Vendor	5 (Equal To)	Vendor Profile ID for the Vendor Contact

Field Filter (Active) Example

XML	JSON
<pre> <FieldFilter> <Field> <ShortName>Active</ShortName> </Field> <FilterType>5</FilterType> <Value>False</Value> </FieldFilter> </pre>	<pre> "filters": [{ "Field": { "ShortName": "Active" }, "FilterType": "5", "Value": "false" }] </pre>

Field Filter (Deleted) Example

XML	JSON
<pre> <FieldFilter> <Field> <ShortName>Deleted</ShortName> </Field> <FilterType>6</FilterType> <Value>True</Value> </FieldFilter> </pre>	<pre> "filters": [{ "Field": { "ShortName": "Deleted" }, "FilterType": "6", "Value": "true" }] </pre>

Field Filter (AccountType) Example

XML	JSON
<pre> <FieldFilter> <Field> <ShortName>AccountType</ShortName> </Field> <FilterType>10002</FilterType> <Value>1 2</Value> </FieldFilter> </pre>	<pre> "filters": [{ "Field": { "ShortName": "AccountType" }, "FilterType": "10002", "Value": "1 2" }] </pre>

Field Filter (Vendor) Example

This example shows how to return Vendor Contacts with non-NAVEX IRM User Accounts.

XML	JSON
<pre> <FieldFilter> <Field> <ShortName>Vendor</ShortName> </Field> <FilterType>5</FilterType> <Value>7</Value> </FieldFilter> </pre>	<pre> "filters": [{ "Field": { "ShortName": "Vendor" }, "FilterType": "5", "Value": "7" }] </pre>

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @GetUserCount.xml
```

```
https://api.lockpath.app:4443/SecurityService/GetUserCount
```

XML REQUEST (GetUserCount.xml)

```
<GetUserCount>
  <filters>
    <FieldFilter>
      <Field>
        <ShortName>AccountType</ShortName>
      </Field>
      <FilterType>5</FilterType>
      <Value>1</Value>
    </FieldFilter>
  </filters>
</GetUserCount>
```

XML RESPONSE

```
<int xmlns="http://schemas.microsoft.com/2003/10/Serialization/">#</int>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X POST -d @GetUserCount.json
```

```
https://api.lockpath.app:4443/SecurityService/GetUserCount
```

JSON REQUEST (GetUserCount.json)

```
[
  {
    "Field": {
      "ShortName": "AccountType"
    },
    "FilterType": "5",
  },
]
```

```
    "Value": "1"
  }
]
```

JSON RESPONSE

```
#
```

CreateUser

Creates a user account.

Permissions: The authentication account must have:

- Administrative Access - Create and Read permissions to Administer - Users

Vendor Contact accounts must also have:

- General Access - Create, Read, and Update permissions to Vendor Profiles
- View and Edit permissions to Vendor Profiles workflow stage
- Vendor Profiles record permissions

URL: `https://[instance name]:[port]/SecurityService/CreateUser`

Method: POST

Input: Various user fields.

Language (integer):

The User object of the GetUser call reveals the language in use in the NAVEX IRM Platform and works in combination with the Preferred Locale feature. When one of the languages with a corresponding locale code is active in the NAVEX IRM Platform, the Preferred Locale field value in My Profile preferences is set for the user. In the NAVEX IRM Platform, the default language is English, and since English has a related locale code, the default language value is "1033."

If an API request returns a language that is not available, or if a language is not active in the instance, the error message "Invalid Language ID" returns.

Languages

The preferred language of the user is stored in the user profile and is revealed in Setup > Platform > Multilingual > Language. You can hover the cursor over the language name in the list to obtain the ID.

The following list includes the language ID, language name, locale ID, and locale name for the languages with corresponding locales. For a list of all languages and language IDs available in the NAVEX IRM Platform, see [Language IDs](#) in this appendix.

Language ID	Language Name	Locale ID	Locale Name
9	English	1033	English (United States)
9	English	2057	English (United Kingdom)
12	French	1036	French (France)

Language ID	Language Name	Locale ID	Locale Name
16	Italian	1040	Italian (Italy)
10	Spanish	2058	Spanish (Mexico)
10	Spanish	3082	Spanish (Spain)

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @CreateUser.xml
```

```
"https://api.lockpath.app:4443/SecurityService/CreateUser"
```

XML REQUEST (CreateUser.xml)

```
<CreateUser>
  <user xmlns:i='http://www.w3.org/2001/XMLSchema-instance'
xmlns:a='http://www.w3.org/2001/XMLSchema'>
    <Username>testuser</Username>
    <Password>P@ssword2!</Password>
    <Active>true</Active>
    <Locked>false</Locked>
    <AccountType>1</AccountType>
    <FirstName>test</FirstName>
    <MiddleName></MiddleName>
    <LastName>user</LastName>
    <Title></Title>
    <Language>1033</Language>
    <EmailAddress>test@user.com</EmailAddress>
    <HomePhone></HomePhone>
    <WorkPhone></WorkPhone>
    <MobilePhone></MobilePhone>
    <Fax></Fax>
    <Manager>
      <Id>12</Id>
    </Manager>
    <Department>
      <Id>10</Id>
    </Department>
    <IsSAML>false</IsSAML>
    <IsLDAP>false</IsLDAP>
    <LDAPDirectory>
```

```

        <Id>1</Id>
    </LDAPDirectory>
    <SecurityConfiguration>
        <Id>1</Id>
    </SecurityConfiguration>
    <APIAccess>false</APIAccess>
    <Groups>
        <Group>
            <Id>5</Id>
        </Group>
    </Groups>
    <SecurityRoles>
        <SecurityRole>
            <Id>1</Id>
        </SecurityRole>
    </SecurityRoles>
</user>
</CreateUser>

```

XML REQUEST (CreateVendorContact.xml)

```

<CreateUser>
    <user xmlns:i='http://www.w3.org/2001/XMLSchema-instance'
        xmlns:a='http://www.w3.org/2001/XMLSchema'>
        <Username>vendor contact</Username>
        <Password>p@ssword!</Password>
        <Active>true</Active>
        <Locked>false</Locked>
        <AccountType>2</AccountType>
        <Vendor>
            <Id>7</Id>
        </Vendor>
        <FirstName>Vendor</FirstName>
        <MiddleName></MiddleName>
    </user>
</CreateUser>

```

```

        <LastName>Contact</LastName>

        <Title>Manager</Title>

        <Language>1033</Language>

        <EmailAddress>test@user.com</EmailAddress>

        <HomePhone></HomePhone>

        <WorkPhone></WorkPhone>

        <MobilePhone></MobilePhone>

        <Fax></Fax>

        <SecurityRoles>
            <SecurityRole>
                <Id>1</Id>
            </SecurityRole>
        </SecurityRoles>
    </user>
</CreateUser>

```

XML RESPONSE (CreateUser.xml)

```

<UserItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
    <Id>1063</Id>
    <FullName>user, test</FullName>
    <Username>testuser</Username>
    <Active>true</Active>
    <Locked>false</Locked>
    <Deleted>false</Deleted>
    <AccountType>1</AccountType>
    <FirstName>test</FirstName>
    <MiddleName/>
    <LastName>user</LastName>
    <Title/>
    <Language>1033</Language>
    <EmailAddress>test@user.com</EmailAddress>
    <HomePhone/>
    <WorkPhone/>

```

```

    <MobilePhone/>

    <Fax/>

    <Manager>
        <Id>12</Id>
        <FullName>Name, User</FullName>
    </Manager>

    <Department>
        <Id>10</Id>
        <Name>Anonymous Incident Analysts</Name>
    </Department>

    <IsSAML>false</IsSAML>

    <IsLDAP>false</IsLDAP>

    <SecurityConfiguration>
        <Id>1</Id>
        <DisplayName>Administrator Configuration</DisplayName>
    </SecurityConfiguration>

    <APIAccess>false</APIAccess>

    <Groups>
        <Group>
            <Id>5</Id>
            <Name>Policy Incident Analysts</Name>
        </Group>
    </Groups>

    <SecurityRoles>
        <SecurityRole>
            <Id>1</Id>
            <Name>Administration</Name>
        </SecurityRole>
    </SecurityRoles>

    <FunctionalRoles/>
</UserItem>

```

XML RESPONSE (CreateVendorContact.xml)

```

<UserItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">

```

```

<Id>1062</Id>

<FullName>Contact, Vendor</FullName>

<Username>vendor contact</Username>

<Active>true</Active>

<Locked>false</Locked>

<Deleted>false</Deleted>

<AccountType>2</AccountType>

<Vendor>
    <Id>7</Id>
    <DisplayName>KC Food Truck</DisplayName>
</Vendor>

<FirstName>Vendor</FirstName>

<MiddleName/>

<LastName>Contact</LastName>

<Title>Manager</Title>

<Language>1033</Language>

<EmailAddress>test@user.com</EmailAddress>

<WorkPhone/>

<MobilePhone/>

<Fax/>

<IsSAML>false</IsSAML>

<IsLDAP>false</IsLDAP>

<SecurityConfiguration>
    <Id>12</Id>
    <DisplayName>Vendor Contacts</DisplayName>
</SecurityConfiguration>

<APIAccess>false</APIAccess>

<VendorComments/>

<Groups/>
</UserItem>

```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X POST -d @CreateUser.json
```

https://api.lockpath.app:4443/SecurityService/CreateUser

JSON REQUEST (CreateUser.json)

```
{
  "Username": "test",
  "Password": "password",
  "Active": true,
  "Locked": false,
  "AccountType": 1,
  "FirstName": "Test",
  "MiddleName": "",
  "LastName": "User",
  "Title": "",
  "Language": 1033,
  "EmailAddress": "test@user.com",
  "HomePhone": "",
  "WorkPhone": "",
  "MobilePhone": "",
  "Fax": "",
  "IsSAML": false,
  "IsLDAP": true,
  "LDAPDirectory": {
    "Id": "1"
  },
  "Manager": {
    "Id": "10"
  },
  "Department": {
    "Id": "10"
  },
  "SecurityConfiguration": {
    "Id": "1"
  },
  "APIAccess": false,
```

```

    "Groups": [
      {
        "Id": "2"
      }
    ],
    "SecurityRoles": [
      {
        "Id": "1"
      }
    ],
    "FunctionalRoles": [
      {
        "Id": "9"
      }
    ]
  ]
}

```

JSON REQUEST (CreateVendorContact.json)

```

{
  "Username": "vuser",
  "Password": "password",
  "Active": true,
  "Locked": false,
  "AccountType": 2,
  "Vendor": {
    "Id": 7
  },
  "FirstName": "Test",
  "MiddleName": "",
  "LastName": "User",
  "Title": "",
  "Language": 1033,
  "EmailAddress": "test@user.com",
  "HomePhone": ""
}

```

```
    "WorkPhone": "",
    "MobilePhone": "",
    "Fax": ""
}
```

JSON RESPONSE (CreateUser.json)

```
{
  "Id": 1067,
  "FullName": "User, Test",
  "Username": "testuser",
  "Active": true,
  "Locked": false,
  "Deleted": false,
  "AccountType": 1,
  "FirstName": "Test",
  "MiddleName": "",
  "LastName": "User",
  "Title": "",
  "Language": 1033,
  "EmailAddress": "test@user.com",
  "HomePhone": "",
  "WorkPhone": "",
  "MobilePhone": "",
  "Fax": "",
  "Manager": {
    "Id": 10,
    "FullName": "Manager"
  },
  "Department": {
    "Id": 10,
    "Name": "Anonymous Incident Analysts"
  },
  "IsSAML": false,
```

```

    "IsLDAP": true,
    "LDAPDirectory": {
      "Id": 1,
      "DisplayName": "QE Domain"
    },
    "SecurityConfiguration": {
      "Id": 1,
      "DisplayName": "Administrator Configuration"
    },
    "APIAccess": false,
    "Groups": [
      {
        "Id": 2,
        "Name": "Exception Analysts"
      }
    ],
    "SecurityRoles": [
      {
        "Id": 1,
        "Name": "Administration"
      }
    ],
    "FunctionalRoles": [
      {
        "Id": 9,
        "DisplayName": "Product Analysis and Design"
      }
    ]
  ]
}

```

JSON RESPONSE (CreateVendorContact.json)

```

{
  "Id": 1068,

```

```

    "FullName": "User, Test",
    "Username": "vuser",
    "Active": true,
    "Locked": false,
    "Deleted": false,
    "AccountType": 2,
    "Vendor": {
        "Id": 7,
        "DisplayName": "KC Food Truck"
    },
    "FirstName": "Test",
    "MiddleName": "",
    "LastName": "User",
    "Title": "",
    "Language": 1033,
    "EmailAddress": "test@user.com",
    "HomePhone": "",
    "WorkPhone": "",
    "MobilePhone": "",
    "Fax": "",
    "IsSAML": false,
    "IsLDAP": false,
    "SecurityConfiguration": {
        "Id": 12,
        "DisplayName": "Vendor Contacts"
    },
    "VendorComments": "",
    "Groups": []
}

```

UpdateUser

Updates a user account.

Permissions: The authentication account must have:

- Administrative Access - Read and Update permissions to Administer - Users

Vendor Contact accounts must also have:

- General Access - Read and Update permissions to Vendor Profiles
- View and Edit permissions to Vendor Profiles workflow stage
- Vendor Profiles record permissions

URL: `https://[instance name]:[port]/SecurityService/UpdateUser`

Method: POST

Input: Various user fields.

CommitNullProperties (boolean) (optional):

Clears all non-required string field values. By default, the setting is false. Use the GetUser call to retrieve current values of the User object and hand UpdateUser the same User object in the request, minus whatever fields you intend to clear. The CommitNullProperties tag must be placed at the end of the User object.

Language (integer):

The User object of the GetUser call reveals the language in use in the NAVEX IRM Platform and works in combination with the Preferred Locale feature. When one of the languages with a corresponding locale code is active in the NAVEX IRM Platform, the Preferred Locale field value in My Profile preferences is set for the user. In the NAVEX IRM Platform, the default language is English, and since English has a related locale code, the default language value is "1033."

If an API request returns a language that is not available, or if a language is not active in the instance, the error message "Invalid Language ID" returns.

Languages

The preferred language of the user is stored in the user profile and is revealed in Setup > Platform > Multilingual > Language. You can hover the cursor over the language name in the list to obtain the ID.

The following list includes the language ID, language name, locale ID, and locale name for the languages with corresponding locales. For a list of all languages and language IDs available in the NAVEX IRM Platform, see Language IDs in the appendix.

Language ID	Language Name	Locale ID	Locale Name
9	English	1033	English (United States)
9	English	2057	English (United Kingdom)
12	French	1036	French (France)
16	Italian	1040	Italian (Italy)
10	Spanish	2058	Spanish (Mexico)
10	Spanish	3082	Spanish (Spain)

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @UpdateUser.xml
```

```
"https://api.lockpath.app:4443/SecurityService/UpdateUser"
```

XML REQUEST (UpdateUser.xml)

```
<UpdateUser>
<user xmlns:i='http://www.w3.org/2001/XMLSchema-instance'
xmlns:a='http://www.w3.org/2001/XMLSchema'>
  <Id>10</Id>
  <Username>test</Username>
  <Password>password</Password>
  <Active>true</Active>
  <Locked>false</Locked>
  <AccountType>1</AccountType>
  <FirstName>test</FirstName>
  <MiddleName></MiddleName>
  <LastName>user</LastName>
  <Title></Title>
  <Language>1033</Language>
  <EmailAddress>test@user.com</EmailAddress>
  <HomePhone></HomePhone>
  <WorkPhone>9132345678</WorkPhone>
  <MobilePhone></MobilePhone>
  <Fax></Fax>
  <IsSAML>false</IsSAML>
  <IsLDAP>true</IsLDAP>
  <LDAPDirectory>
    <Id>1</Id>
  </LDAPDirectory>
  <Manager></Manager>
  <Department></Department>
```

```

    <SecurityConfiguration>
        <Id>1</Id>
    </SecurityConfiguration>
    <APIAccess>true</APIAccess>
    <Groups>
        <Group>
            <Id>2</Id>
        </Group>
    </Groups>
    <CommitNullProperties>true</CommitNullProperties>
</user>
</UpdateUser>

```

XML RESPONSE

```

<UserItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
    <Id>10</Id>
    <FullName>testuser</FullName>
    <Username>test</Username>
    <Active>true</Active>
    <Locked>false</Locked>
    <Deleted>false</Deleted>
    <AccountType>1</AccountType>
    <FirstName>test</FirstName>
    <MiddleName/>
    <LastName>user</LastName>
    <Title/>
    <Language>1033</Language>
    <EmailAddress>test@user.com</EmailAddress>
    <HomePhone/>
    <WorkPhone/>9131234567</WorkPhone>
    <MobilePhone/>
    <Fax/>
    <IsSAML>false</IsSAML>

```

```

<IsLDAP>true</IsLDAP>

<LDAPDirectory>
  <Id>1</Id>
  <DisplayName>QE Domain</DisplayName>
</LDAPDirectory>

<SecurityConfiguration>
  <Id>1</Id>
  <DisplayName>Administrator Configuration</DisplayName>
</SecurityConfiguration>

<APIAccess>true</APIAccess>

<Groups>
  <Group>
    <Id>2</Id>
    <Name>Exception Analysts</Name>
  </Group>
</Groups>

<SecurityRoles>
  <SecurityRole>
    <Id>1</Id>
    <Name>Administration</Name>
  </SecurityRole>
</SecurityRoles>

<FunctionalRoles/>
</UserItem>

```

JSON REQUEST (cURL)

```

curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @UpdateUser.json
https://api.lockpath.app:4443/SecurityService/UpdateUser

```

JSON REQUEST (UpdateUser.json)

```

{
  "Id": "10",
  "Username": "test",

```

```

    "Password": "password",
    "Active": true,
    "Locked": false,
    "AccountType": 1,
    "FirstName": "Test",
    "MiddleName": "",
    "LastName": "User",
    "Title": "",
    "Language": 1033,
    "EmailAddress": "test@user.com",
    "HomePhone": "",
    "WorkPhone": "9132345678",
    "MobilePhone": "",
    "Fax": "",
    "IsSAML": false,
    "IsLDAP": true,
    "LDAPDirectory": {
        "Id": "1"
    },
    "Manager": "",
    "Department": "",
    "SecurityConfiguration": {
        "Id": "1"
    },
    "APIAccess": true,
    "CommitNullProperties": "true"
}

```

JSON RESPONSE

```

{
    "Id": 10,
    "FullName": "User, Test",
    "Username": "test",

```

```
"Active": true,
"Locked": false,
"Deleted": false,
"AccountType": 1,
"FirstName": "Test",
"MiddleName": "",
"LastName": "User",
"Title": "",
"Language": 1033,
"EmailAddress": "test@user.com",
"HomePhone": "",
"WorkPhone": "9132345678",
"MobilePhone": "",
"Fax": "",
"IsSAML": false,
"IsLDAP": true,
"LDAPDirectory": {
  "Id": 1,
  "DisplayName": "QE Domain"
},
"SecurityConfiguration": {
  "Id": 1,
  "DisplayName": "Administrator Configuration"
},
"APIAccess": true,
"Groups": [],
"SecurityRoles": [
  {
    "Id": 1,
    "Name": "Administration"
  }
],
```

```
"FunctionalRoles": []  
}
```

DeleteUser

Deletes a user account.

Permissions: The authentication account must have:

- Administrative Access - Read and Delete permissions to Administer - Users

Vendor Contact accounts must also have:

- General Access - Read and Delete permissions to Vendor Profiles
- View permissions to Vendor Profiles workflow stage
- Vendor Profiles record permissions

URL: `https://[instance name]:[port]/SecurityService/DeleteUser`

Method: DELETE

Input: Id (integer): The Id of the user account.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X DELETE -d
"<DeleteUser><id>10</id></DeleteUser>"
```

```
"https://api.lockpath.app:4443/SecurityService/DeleteUser"
```

XML RESPONSE

```
<boolean xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</boolean>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
DELETE -d "10"
```

```
https://api.lockpath.app:4443/SecurityService/DeleteUser
```

JSON RESPONSE

```
true
```

GetGroup

Returns all supporting fields for a given group.

Permissions: The authentication account must have:

- Administrative Access - Read permissions to Administer - Groups

URL: `https://[instance name]:[port]/SecurityService/GetGroup`

Method: GET

Input: Id: The Id of the given group.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
"https://api.lockpath.app:4443/SecurityService/GetGroup?id=2"
```

XML RESPONSE

```
<GroupItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>2</Id>
  <Name>Anonymous Incident Analysts</Name>
  <Description/>
  <BusinessUnit>false</BusinessUnit>
  <SecurityRoles/>
  <Users>
    <User>
      <Id>6</Id>
      <FullName>Admin, Lockpath</FullName>
    </User>
    <User>
      <Id>27</Id>
      <FullName>User, Test</FullName>
    </User>
  </Users>
  <ChildGroups/>
  <ParentGroups/>
</GroupItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json"
https://api.lockpath.app:4443/SecurityService/GetGroup?id=2
```

JSON RESPONSE

```
{
  "Id": 2,
  "Name": "Anonymous Incident Analysts",
  "Description": "",
  "BusinessUnit": false,
  "SecurityRoles": [],
  "Users": [
    {
      "Id": 6,
      "FullName": "Admin, Lockpath"
    },
    {
      "Id": 27,
      "FullName": "User, Test"
    }
  ],
  "ChildGroups": [],
  "ParentGroups": []
}
```

GetGroups

Returns the ID and Name for groups. You can filter the list to return only the groups meeting selected criteria.

Permissions: The authentication account must have:

- Administrative Access - Read permissions to Administer - Groups

URL: `https://[instance name]:[port]/SecurityService/GetGroups`

Method: POST

Input: `pageIndex (integer)`: The index of the page of result to return. Must be ≥ 0 .
`pageSize (integer)`: The size of the page results to return. Must be ≥ 1 .
`FieldFilter (various) (optional)`: The filter parameters the groups must meet to be included.

Field Filters

Use filters to return only the users meeting the selected criteria.

Field ShortName	FilterType	Value
Active	5 (Equal To)	True, False
	6 (NotEqualTo)	
Deleted	5 (Equal To)	True, False
	6 (NotEqualTo)	
AccountType	5 (Equal To)	1 (Full User), 2 (Vendor, VendorContact), 4 (Awareness User)
	6 (NotEqualTo)	
	10002 (Contains Any)	

Filter Example

XML	JSON
<pre> <filters> <FieldFilter> <FieldPath> <Field> <ShortName>Active</ShortName> </Field> </FieldPath> <FilterType>5</FilterType> <Value>False</Value> </FieldFilter> <FieldFilter> <Field> <ShortName>Deleted</ShortName> </Field> <FilterType>6</FilterType> <Value>True</Value> </FieldFilter> <FieldFilter> <Field> <ShortName>AccountType</ShortName> </Field> <FilterType>10002</FilterType> <Value>1 4</Value> </FieldFilter> </filters> </pre>	<pre> { "filters": { "FieldFilter": [{ "FieldPath": { "Field": { "ShortName": "Active" } }, "FilterType": 5, "Value": "False" }, { "Field": { "ShortName": "Deleted" }, "FilterType": 6, "Value": "True" }, { "Field": { "ShortName": "AccountType" }, "FilterType": 10002, "Value": "1 4" }] } } </pre>

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @GetGroups.xml
```

```
https://api.lockpath.app:4443/SecurityService/GetGroups
```

XML REQUEST (GetGroups.xml)

```
<GetGroups>
  <pageIndex>0</pageIndex>
  <pageSize>2</pageSize>
  <filters>
    <FieldFilter>
      <FieldPath>
        <Field>
          <ShortName>BusinessUnit</ShortName>
        </Field>
      </FieldPath>
      <FilterType>5</FilterType>
      <Value>False</Value>
    </FieldFilter>
  </filters>
</GetGroups>
```

XML RESPONSE

```
<GroupList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Group>
    <Id>10</Id>
    <Name>Anonymous Incident Analysts</Name>
  </Group>
  <Group>
    <Id>6</Id>
    <Name>Business Continuity Plan Approvers</Name>
  </Group>
</GroupList>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @GetGroups.json

https://api.lockpath.app:4443/SecurityService/GetGroups
```

JSON REQUEST (GetGroups.json)

```
{
  "pageIndex": "0",
  "pageSize": "2",
  "filters":
  [
    {
      "Field":
      {
        "ShortName": "BusinessUnit"
      },
      "FilterType": "5",
      "Value": "False"
    }
  ]
}
```

JSON RESPONSE

```
[
  {
    "Id": 10,
    "Name": "Anonymous Incident Analysts"
  },
  {
    "Id": 6,
    "Name": "Business Continuity Plan Approvers"
  }
]
```

CreateGroup

Creates a group.

Permissions: The authentication account must have:

- Administrative Access - Read and Create permissions to Administer - Groups

URL: `https://[instance name]:[port]/SecurityService/CreateGroup`

Method: POST

Input: Various group fields.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @CreateGroup.xml
```

`https://api.lockpath.app:4443/SecurityService/CreateGroup`

XML REQUEST (CreateGroup.xml)

```
<CreateGroup>

<group xmlns:i='http://www.w3.org/2001/XMLSchema-instance'
xmlns:a='http://www.w3.org/2001/XMLSchema'>

    <Name>Business Owners</Name>

    <Description>Business Owners Group</Description>

    <BusinessUnit>false</BusinessUnit>

    <Users>

        <User>

            <Id>1027</Id>

        </User>

        <User>

            <Id>6</Id>

        </User>

    </Users>

    <ChildGroups></ChildGroups>

    <ParentGroups></ParentGroups>

</group>

</CreateGroup>
```

XML RESPONSE

```
<GroupItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>20</Id>
  <Name>Business Owners</Name>
  <Description>Business Owners Group</Description>
  <BusinessUnit>>false</BusinessUnit>
  <SecurityRoles/>
  <Users>
    <User>
      <Id>1027</Id>
      <FullName>User, Test</FullName>
    </User>
  </Users>
  <ChildGroups/>
  <ParentGroups/>
</GroupItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @CreateGroup.json
```

```
https://api.lockpath.app:4443/SecurityService/CreateGroup
```

JSON REQUEST (CreateGroup.json)

```
{
  "Id": 21,
  "Name": "Government",
  "Description": "Government Group",
  "BusinessUnit": false,
  "SecurityRoles": "",
  "Users": [{
    "Id": 1027
  }],
  "ChildGroups": "",
  "ParentGroups": ""
}
```

```
}
```

JSON RESPONSE

```
{
  "Id": 22,
  "Name": "Government",
  "Description": "Government Group",
  "BusinessUnit": false,
  "SecurityRoles": [],
  "Users": [
    {
      "Id": 1027,
      "FullName": "User, Test"
    }
  ],
  "ChildGroups": [],
  "ParentGroups": []
}
```

UpdateGroup

Updates a group.

Permissions: The authentication account must have:

- Administrative Access - Read and Update permissions to Administer - Groups

URL: `https://[instance name]:[port]/SecurityService/UpdateGroup`

Method: POST

Input: Various group fields.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @UpdateGroup.xml
```

```
https://api.lockpath.app:4443/SecurityService/UpdateGroup
```

XML REQUEST (UpdateGroup.xml)

```
<UpdateGroup>
  <group xmlns:i='http://www.w3.org/2001/XMLSchema-instance'
    xmlns:a='http://www.w3.org/2001/XMLSchema'>
    <Id>6</Id>
    <Name>API Group</Name>
    <Description>This is an update.</Description>
    <BusinessUnit>>false</BusinessUnit>
    <Users>
      <User>
        <Id>10</Id>
      </User>
      <User>
        <Id>11</Id>
      </User>
    </Users>
    <ParentGroups>
      <Group>
        <Id>2</Id>
```

```

        </Group>
        <Group>
            <Id>3</Id>
        </Group>
    </ParentGroups>
</group>
</UpdateGroup>

```

XML RESPONSE

```

<GroupItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
    <Id>6</Id>
    <Name>API Group</Name>
    <Description>This is an update.</Description>
    <BusinessUnit>>false</BusinessUnit>
    <SecurityRoles/>
    <Users>
        <User>
            <Id>10</Id>
            <FullName>User, Test</FullName>
        </User>
        <User>
            <Id>11</Id>
            <FullName>User, API</FullName>
        </User>
    </Users>
    <ChildGroups/>
    <ParentGroups>
        <Group>
            <Id>2</Id>
            <Name>Exception Analysts</Name>
        </Group>
        <Group>
            <Id>3</Id>

```

```
        <Name>Information Security Incident Analysts</Name>

    </Group>

</ParentGroups>

</GroupItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @UpdateGroup.json
https://api.lockpath.app:4443/SecurityService/UpdateGroup
```

JSON REQUEST (UpdateGroup.json)

```
{
  "Id": 6,
  "Name": "API Group",
  "Description": "Another API update.",
  "BusinessUnit": false,
  "Users": [
    {
      "Id": 10
    },
    {
      "Id": 11
    }
  ],
  "ParentGroups": [
    {
      "Id": 2
    },
    {
      "Id": 3
    }
  ]
}
```

JSON RESPONSE

```
{
  "Id": 6,
  "Name": "API Group",
  "Description": "Another API update.",
  "BusinessUnit": false,
  "SecurityRoles": [],
  "Users": [
    {
      "Id": 10,
      "FullName": "User, Test"
    },
    {
      "Id": 11,
      "FullName": "User, API"
    }
  ],
  "ChildGroups": [],
  "ParentGroups": [
    {
      "Id": 2,
      "Name": "Exception Analysts"
    },
    {
      "Id": 3,
      "Name": "Information Security Incident Analysts"
    }
  ]
}
```

DeleteGroup

Deletes a group.

Permissions: The authentication account must have:

- Administrative Access - Read and Delete permissions to Administer - Groups

URL: `https://[instance name]:[port]/SecurityService/DeleteGroup`

Method: DELETE

Input: Id (integer): The Id of the group.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X DELETE -d "<DeleteGroup><id>10</id></DeleteGroup>"
```

`https://api.lockpath.app:4443/SecurityService/DeleteGroup`

XML RESPONSE

```
<boolean xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</boolean>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X DELETE -d "10"
```

`https://api.lockpath.app:4443/SecurityService/DeleteGroup`

JSON RESPONSE

```
true
```

3: Component Services API

This chapter provides a list of the available methods for retrieving, updating, and deleting data from the NAVEX IRM Platform.

GetComponent	72
GetComponentList	74
GetComponentByAlias	76
GetFieldList	78
GetField	81
GetAvailableLookupRecords	83
GetRecord	86
GetRecords	88
GetRecordCount	92
GetDetailRecord	96
GetDetailRecords	99
GetLookupReportColumnFields	111
GetRecordAttachment	113
GetRecordAttachments	115
GetWorkflow	117
GetWorkflows	121
TransitionRecord	123
VoteRecord	125
CreateRecord	127
UpdateRecord	134
UpdateRecordAttachments	138
DeleteRecord	142
DeleteRecordAttachments	144
ImportFile	148

GetComponent

Retrieves a component specified by its ID. A component is a user-defined data object such as a custom content table. The component ID may be found by using GetComponentList.

Permissions: The authentication account must have:

- General Access - Read permissions to the selected component

URL: `https://[instance name]:[port]/ComponentService/GetComponent?id={ID}`

Method: GET

Input: Id (integer): The Id of the desired component.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetComponent?id=10021
```

XML RESPONSE

```
<ComponentItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>10021</Id>
  <Name>Incident Reports</Name>
  <SystemName>IncidentReports</SystemName>
  <ShortName>IncidentReports</ShortName>
</ComponentItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/ComponentService/GetComponent?id=10021
```

JSON RESPONSE

```
{  
  "Id": 10021,  
  "Name": "Incident Reports",  
  "SystemName": "IncidentReports",  
  "ShortName": "IncidentReports"  
}
```

GetComponentList

Returns a complete list of all components available to the user based on account permissions. No input elements are used. The list will be ordered in ascending alphabetical order of the component name.

Permissions: The authentication account must have:

- General Access - Read permissions to the selected components

URL: `https://[instance name]:[port]/ComponentService/GetComponentList`

Method: GET

Input: No inputs allowed.

Examples

In the samples, an ellipse (...) indicates an incomplete list of items returned.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetComponentList
```

XML RESPONSE

```
<ComponentList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Component>
    <Id>10003</Id>
    <Name>Device Types</Name>
    <SystemName>DeviceTypes</SystemName>
    <ShortName>DeviceTypes</ShortName>
  </Component>
  <Component>
    <Id>10001</Id>
    <Name>Devices</Name>
    <SystemName>Devices</SystemName>
    <ShortName>Devices</ShortName>
  </Component>
  ...
</ComponentList>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/ComponentService/GetComponentList
```

JSON RESPONSE

```
[  
  {  
    "Id": "10003",  
    "Name": "Device Types",  
    "SystemName": "DeviceTypes",  
    "ShortName": "DeviceTypes"  
  },  
  {  
    "Id": "10001",  
    "Name": "Devices",  
    "SystemName": "Devices",  
    "ShortName": "Devices"  
  }  
  ...  
]
```

GetComponentByAlias

Retrieves a component specified by its Alias. A component is a user-defined data object such as a custom content table. The component Alias may be found by using GetComponentList.

Permissions: The authentication account must have:

- General Access - Read permissions to the specific component

URL: `https://[instance name]:[port]/ComponentService/GetComponentByAlias?alias={ALIAS}`

Method: GET

Input: Alias (string): The alias (shortname) of the component.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetComponentByAlias?alias=IncidentReports
```

XML RESPONSE

```
<ComponentItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>10021</Id>
  <Name>Incident Reports</Name>
  <SystemName>IncidentReports</SystemName>
  <ShortName>IncidentReports</ShortName>
</ComponentItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/ComponentService/GetComponentByAlias?alias=IncidentReports
```

JSON RESPONSE

```
{  
  "Id": 10021,  
  "Name": "Incident Reports",  
  "SystemName": "IncidentReports",  
  "ShortName": "IncidentReports"  
}
```

GetFieldList

Retrieves a detailed field list for a component specified by its ID. The component ID may be found by using GetComponentList. Assessments field type are not visible in this list.

Permissions: The authentication account must have:

- General Access - Read permissions for the selected component.
- View field permissions. Fields to which the account does not have access are not returned.

URL: `https://[instance name]:[port]/ComponentService/GetFieldList?componentId={COMPONENTID}`

Method: GET

Input: componentId (integer): The Id of the desired component.

Examples

In the samples, an ellipse (...) indicates an incomplete list of items returned.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetFieldList?componentId=10001
```

XML RESPONSE

```
<FieldList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Field>
    <Id>9</Id>
    <Name>Acquisition Cost</Name>
    <SystemName>Cost</SystemName>
    <ShortName>Cost</ShortName>
    <ReadOnly>>false</ReadOnly>
    <Required>>false</Required>
    <FieldType>2</FieldType>
    <Precision>15</Precision>
    <Scale>2</Scale>
    <OneToMany>>false</OneToMany>
    <MatrixRows/>
  </Field>
  <Field>
    <Id>10</Id>
    <Name>Acquisition Date</Name>
```

```
<SystemName>AcquisitionDate</SystemName>
<ShortName>AcquisitionDate</ShortName>
<ReadOnly>false</ReadOnly>
<Required>false</Required>
<Nullable>false</Nullable>
<FieldType>3</FieldType>
<OneToMany>false</OneToMany>
<MatrixRows/>
</Field>
...
</FieldList>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/ComponentService/GetFieldList?componentId=10001
```

JSON RESPONSE

```
[  
  {  
    "Id": 9,  
    "Name": "Acquisition Cost",  
    "SystemName": "Cost",  
    "ShortName": "Cost",  
    "ReadOnly": false,  
    "Required": false,  
    "FieldType": 2,  
    "Precision": 15,  
    "Scale": 2,  
    "OneToMany": false,  
    "MatrixRows": []  
  },  
  {  
    "Id": 10,  
    "Name": "Acquisition Date",  
    "SystemName": "AcquisitionDate",  
    "ShortName": "AcquisitionDate",  
    "ReadOnly": false,  
    "Required": false,  
    "FieldType": 3,  
    "OneToMany": false,  
    "MatrixRows": []  
  },  
  ...  
]  
}
```

GetField

Retrieves details for a field within a component. The field ID may be found by using GetFieldList.

Permissions: The authentication account must have:

- General Access - Read permission to the component and field

URL: `https://[instance name]:[port]/ComponentService/GetField?id={FIELDID}`

Method: GET

Input: Id (integer): The field Id for the individual field within the component.

Examples

GetField returns a FieldType. The application uses several field types each represented by a unique ID.

Field Types			
FieldType ID	Attribute	FieldType ID	Attribute
1	Text	6	Master/Detail
2	Numeric	7	Matrix
3	Date	8	Documents
4	IP Address	9	Assessments
5	Lookup	10	Yes/No

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetField?id=23
```

XML RESPONSE

```
<?xml version="1.0"?>
<FieldItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>23</Id>
  <Name>Device Type</Name>
  <SystemName>DeviceType</SystemName>
  <ShortName>DeviceType</ShortName>
  <ReadOnly>>false</ReadOnly>
  <Required>>false</Required>
  <FieldType>5</FieldType>
  <OneToMany>>false</OneToMany>
  <MatrixRows/>
</FieldItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"
https://api.lockpath.app:4443/ComponentService/GetField?Id=23
```

JSON RESPONSE

```
{
  "Id": 23,
  "Name": "Device Type",
  "SystemName": "DeviceType",
  "ShortName": "DeviceType",
  "ReadOnly": false,
  "Required": false,
  "FieldType": 5,
  "OneToMany": false,
  "MatrixRows": []
}
```

GetAvailableLookupRecords

Retrieves records that are available for population for a lookup field.

Permissions: The authentication account must have:

- Create and Read access to the component that contains the lookup field if no recordId is supplied
- Read and Edit access to the component that contains the lookup field if a recordId is supplied
- Read and Edit access to the lookup field
- Read access to the recordId
- Read access to any lookup records

URL: `https://[instance-name]:[port]/ComponentService/GetAvailableLookupRecords`

Method: POST

Input:

<code>fieldId (integer):</code>	The ID of the desired component.
<code>pageIndex (integer):</code>	The index of the page of result to return. Must be ≥ 0 .
<code>pageSize (integer):</code>	The size of the page results to return. Must be ≥ 1 .
<code>recordId (integer) (optional):</code>	The ID of the record for which retrieving lookup records.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @GetAvailableLookupRecords.xml "https://api.lockpath.app:4443/ComponentService/GetAvailableLookupRecords"
```

XML REQUEST (GetAvailableLookupRecords.xml)

```
<GetAvailableLookupRecords>
  <fieldId>23</fieldId>
  <pageIndex>0</pageIndex>
  <pageSize>2</pageSize>
  <recordId>1</recordId>
</GetAvailableLookupRecords>
```

XML RESPONSE

```
<ArrayOfDynamicRecordItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <DynamicRecordItem>
    <Id>2</Id>
    <DisplayName>Mobile Device</DisplayName>
    <FieldValues/>
  </DynamicRecordItem>
  <DynamicRecordItem>
    <Id>4</Id>
    <DisplayName>Laptop</DisplayName>
    <FieldValues/>
  </DynamicRecordItem>
</ArrayOfDynamicRecordItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @GetAvailableLookupRecords.json
```

```
"https://api.lockpath.app:4443/ComponentService/GetAvailableLookupRecords"
```

JSON REQUEST (GetAvailableLookupRecords.json)

```
{
  "fieldId": "23",
  "pageIndex": "0",
  "pageSize": "2",
  "recordId": "1"
}
```

JSON RESPONSE

```
[
  {
    "Id": 2,
    "DisplayName": "Mobile Device",
    "FieldValues": []
  },
  {
    "Id": 4,
    "DisplayName": "Laptop",
    "FieldValues": []
  }
]
```

GetRecord

Returns the set of fields for a given record for which you have permissions within a component. The field keys can be matched with the data from the GetField method to obtain the name of the field.

Permissions: The authentication account must have General Access - Read permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/GetRecord?componentId={COMPONENTID}&recordId={RECORDID}`

Method: GET

Input: `componentId (integer):` The Id of the desired component.

`recordId (integer):` The Id for the individual record within the component.

Examples

In the samples, an ellipse (...) indicates an incomplete list of items returned.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetRecord?componentId=10001&recordId=1
```

XML RESPONSE

```
<DynamicRecordItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>1</Id>
  <DisplayName>1</DisplayName>
  <FieldValues>
    <KeyValuePair>
      <key>3</key>
      <value i:nil="true"/>
    </KeyValuePair>
    <KeyValuePair>
      <key>4</key>
      <value i:nil="true"/>
    </KeyValuePair>
    ...
  </FieldValues>
</DynamicRecordItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
"https://api.lockpath.app:4443/ComponentService/GetRecord?componentId=10001&recordId=1"
```

JSON RESPONSE

```
{  
  "Id": "1",  
  "DisplayName": "1",  
  "FieldValues": [{  
    "Key": 3,  
    "Value": null  
  },  
  {  
    "Key": 4,  
    "Value": null  
  }  
  ...  
]  
}
```

GetRecords

Returns an array of records within a chosen component. You can use filters to return only the records meeting selected criteria.

Permissions: The authentication account must have General Access - Read permission to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/GetRecords`

Method: POST

Input:	<code>componentId (integer):</code>	The Id of the desired component.
	<code>pageIndex (integer):</code>	The index of the page of result to return. Must be ≥ 0 .
	<code>pageSize (integer):</code>	The size of the page results to return. Must be ≥ 1 .

filters (various) (optional):

The filter parameters the records must meet to be counted.

A variety of filters are available, and an unlimited number of search criteria may be applied to each transaction that supports filtering. Search filters are composed of a field path, filter type, and a value.

Element	Description																																																												
FieldPath	Describes the field ID for the column that the records will be filtered on. If the value is stored in the component directly, only one Field Path variable is needed. However, if the value is a lookup to another component, an additional Field Path variable will be required with the column value where the data resides. Field Path variables can be added as many as necessary to provide the correct path to the data.																																																												
FilterType	Describes the ID for the filter being applied. For example, the FilterType for Is Null is 15. If the FilterType would exclude the entry of a value like Is Empty (13) or Is Not Null (16), the <Value> tags should be removed from the request. <table><tr><th colspan="4">Filter Types</th></tr><tr><th>ID</th><th>Option</th><th>ID</th><th>Option</th></tr><tr><td>1</td><td>Contains</td><td>9</td><td>≥</td></tr><tr><td>2</td><td>Excludes</td><td>10</td><td>≤</td></tr><tr><td>3</td><td>Starts With</td><td>11</td><td>Between</td></tr><tr><td>4</td><td>Ends With</td><td>12</td><td>Not Between</td></tr><tr><td>5</td><td>=</td><td>13</td><td>Is Empty</td></tr><tr><td>6</td><td><></td><td>14</td><td>Is Not Empty</td></tr><tr><td>7</td><td>></td><td>15</td><td>Is Null</td></tr><tr><td>8</td><td><</td><td>16</td><td>Is Not Null</td></tr><tr><td>10001</td><td colspan="3">Offset</td></tr><tr><td>10002</td><td colspan="3">Contains Any (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10003</td><td colspan="3">Contains Only (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10004</td><td colspan="3">Contains None (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10005</td><td colspan="3">Contains At Least (applies only to One-to-Many Lookup fields)</td></tr></table>	Filter Types				ID	Option	ID	Option	1	Contains	9	≥	2	Excludes	10	≤	3	Starts With	11	Between	4	Ends With	12	Not Between	5	=	13	Is Empty	6	<>	14	Is Not Empty	7	>	15	Is Null	8	<	16	Is Not Null	10001	Offset			10002	Contains Any (applies only to One-to-Many Lookup fields)			10003	Contains Only (applies only to One-to-Many Lookup fields)			10004	Contains None (applies only to One-to-Many Lookup fields)			10005	Contains At Least (applies only to One-to-Many Lookup fields)		
Filter Types																																																													
ID	Option	ID	Option																																																										
1	Contains	9	≥																																																										
2	Excludes	10	≤																																																										
3	Starts With	11	Between																																																										
4	Ends With	12	Not Between																																																										
5	=	13	Is Empty																																																										
6	<>	14	Is Not Empty																																																										
7	>	15	Is Null																																																										
8	<	16	Is Not Null																																																										
10001	Offset																																																												
10002	Contains Any (applies only to One-to-Many Lookup fields)																																																												
10003	Contains Only (applies only to One-to-Many Lookup fields)																																																												
10004	Contains None (applies only to One-to-Many Lookup fields)																																																												
10005	Contains At Least (applies only to One-to-Many Lookup fields)																																																												
Value	Describes the comparison that the column will be measured against. FilterType 1 applies only to one-to-many lookup fields. For the FilterType 11 and 12, two values are required that are delimited by a pipe () symbol. For the FilterTypes 13 - 16 no values are required.																																																												
SearchCriteriaItem	Describes a single filter. By combining FieldPath, Filter, and Value, a complete search filter is created for the desired parameters.																																																												

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d
@GetRecordsInput.xml
"https://api.lockpath.app:4443/ComponentService/GetRecords"
```

XML REQUEST (GetRecordsInput.xml)

```
<GetRecords>
  <componentId>10001</componentId>
  <pageIndex>0</pageIndex>
  <pageSize>2</pageSize>
  <filters>
    <SearchCriteriaItem>
      <FieldPath>
        <int>11</int>
      </FieldPath>
      <FilterType>16</FilterType>
      <Value></Value>
    </SearchCriteriaItem>
  </filters>
</GetRecords>
```

XML RESPONSE

```
<ArrayOfDynamicRecordItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <DynamicRecordItem>
    <Id>1</Id>
    <DisplayName>192.168.1.84</DisplayName>
    <FieldValues/>
  </DynamicRecordItem>
  <DynamicRecordItem>
    <Id>2</Id>
    <DisplayName>192.168.1.69</DisplayName>
    <FieldValues/>
  </DynamicRecordItem>
</ArrayOfDynamicRecordItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X POST -d @GetRecordsInput.json
```

```
"https://api.lockpath.app:4443/ComponentService/GetRecords"
```

JSON REQUEST (GetRecordsInput.json)

```
{
  "componentId": "10001",
  "pageIndex": "0",
  "pageSize": "2",
  "filters": [
    {
      "FieldPath": [
        11
      ],
      "FilterType": "16",
      "Value": ""
    }
  ]
}
```

JSON RESPONSE

```
[
  {
    "Id": 1,
    "DisplayName": "192.168.1.84",
    "FieldValues": []
  },
  {
    "Id": 2,
    "DisplayName": "192.168.1.69",
    "FieldValues": []
  }
]
```

GetRecordCount

Returns the number of records in a given component. You can use filters to return the count of records meeting selected criteria.

Permissions: The authentication account must have Read General Access permission to:

- Selected component
- Applicable records
- Applicable records in the component (table)

URL: `https://[instance name]:[port]/ComponentService/GetRecordCount`

Method: POST

Input: `componentId (integer):` The Id of the desired component.

filters (various):

The filter parameters the records must meet to be counted.

A variety of filters are available, and an unlimited number of search criteria may be applied to each transaction that supports filtering. Search filters are composed of a field path, filter type, and a value.

Element	Description																																																												
FieldPath	Describes the field ID for the column that the records will be filtered on. If the value is stored in the component directly, only one Field Path variable is needed. However, if the value is a lookup to another component, an additional Field Path variable will be required with the column value where the data resides. Field Path variables can be added as many as necessary to provide the correct path to the data.																																																												
FilterType	Describes the ID for the filter being applied. For example, the FilterType for Is Null is 15. If the FilterType would exclude the entry of a value like Is Empty (13) or Is Not Null (16), the <Value> tags should be removed from the request. <table><tr><th colspan="4">Filter Types</th></tr><tr><th>ID</th><th>Option</th><th>ID</th><th>Option</th></tr><tr><td>1</td><td>Contains</td><td>9</td><td>≥</td></tr><tr><td>2</td><td>Excludes</td><td>10</td><td>≤</td></tr><tr><td>3</td><td>Starts With</td><td>11</td><td>Between</td></tr><tr><td>4</td><td>Ends With</td><td>12</td><td>Not Between</td></tr><tr><td>5</td><td>=</td><td>13</td><td>Is Empty</td></tr><tr><td>6</td><td><></td><td>14</td><td>Is Not Empty</td></tr><tr><td>7</td><td>></td><td>15</td><td>Is Null</td></tr><tr><td>8</td><td><</td><td>16</td><td>Is Not Null</td></tr><tr><td>10001</td><td colspan="3">Offset</td></tr><tr><td>10002</td><td colspan="3">Contains Any (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10003</td><td colspan="3">Contains Only (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10004</td><td colspan="3">Contains None (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10005</td><td colspan="3">Contains At Least (applies only to One-to-Many Lookup fields)</td></tr></table>	Filter Types				ID	Option	ID	Option	1	Contains	9	≥	2	Excludes	10	≤	3	Starts With	11	Between	4	Ends With	12	Not Between	5	=	13	Is Empty	6	<>	14	Is Not Empty	7	>	15	Is Null	8	<	16	Is Not Null	10001	Offset			10002	Contains Any (applies only to One-to-Many Lookup fields)			10003	Contains Only (applies only to One-to-Many Lookup fields)			10004	Contains None (applies only to One-to-Many Lookup fields)			10005	Contains At Least (applies only to One-to-Many Lookup fields)		
Filter Types																																																													
ID	Option	ID	Option																																																										
1	Contains	9	≥																																																										
2	Excludes	10	≤																																																										
3	Starts With	11	Between																																																										
4	Ends With	12	Not Between																																																										
5	=	13	Is Empty																																																										
6	<>	14	Is Not Empty																																																										
7	>	15	Is Null																																																										
8	<	16	Is Not Null																																																										
10001	Offset																																																												
10002	Contains Any (applies only to One-to-Many Lookup fields)																																																												
10003	Contains Only (applies only to One-to-Many Lookup fields)																																																												
10004	Contains None (applies only to One-to-Many Lookup fields)																																																												
10005	Contains At Least (applies only to One-to-Many Lookup fields)																																																												
Value	Describes the comparison that the column will be measured against. FilterType 1 applies only to one-to-many lookup fields. For the FilterType 11 and 12, two values are required that are delimited by a pipe () symbol. For the FilterTypes 13 - 16 no values are required.																																																												
SearchCriteriaItem	Describes a single filter. By combining FieldPath, Filter, and Value, a complete search filter is created for the desired parameters.																																																												

Examples

In the XML example, the search is looking for values that do not have an empty address field. Because of this, the value parameter is left blank because IsNotEmpty is not comparing to any set value.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d
@GetRecordCount.xml

"https://api.lockpath.app:4443/ComponentService/GetRecordCount"
```

XML REQUEST (GetRecordCount.xml)

```
<GetRecordCount>

<componentId>10001</componentId>

  <filters>
    <SearchCriteriaItem>
      <FieldPath>
        <int>11</int>
      </FieldPath>
      <FilterType>16</FilterType>
    </SearchCriteriaItem>
  </filters>
</GetRecordCount>
```

XML RESPONSE

```
<int xmlns="http://schemas.microsoft.com/2003/10/Serialization/">#</int>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X  
POST -d @GetRecordCount.json  
  
"https://api.lockpath.app:4443/ComponentService/GetRecordCount"
```

JSON REQUEST (GetRecordCount.json)

```
{  
  "componentId": "10001",  
  "filters": [  
    {  
      "FieldPath": [  
        11  
      ],  
      "FilterType": 15,  
      "Value": ""  
    }  
  ]  
}
```

JSON RESPONSE

```
#
```

GetDetailRecord

Retrieves record information based on the provided component ID and record ID, with lookup field report details. Using the optional boolean parameter "embedRichTextImages" you can return records with images contained in rich text fields.

Permissions: The authentication account must have General Access - Read permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/GetDetailRecord?componentId={COMPONENTID}&recordId={RECORDID}&embedRichTextImages=true`

Method: GET

Input:

<code>componentId (integer):</code>	The Id of the desired component.
<code>recordId (integer):</code>	The Id for the individual record within the component.
<code>embedRichTextImages (boolean) (optional):</code>	Include images contained in rich text fields.

Examples

In the samples, an ellipse (...) indicates an incomplete list of items returned.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
```

```
https://api.lockpath.app:4443/ComponentService/GetDetailRecord?componentId=10001&recordId=114&embedRichTextImages=true
```

XML RESPONSE

```
<DynamicRecordItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>114</Id>
  <DisplayName>114</DisplayName>
  <FieldValues>
    ...
    <KeyValuePair>
      <key>34</key>
      <value i:type="DynamicRecordList">
        <Record>
          <Id>55</Id>
```

```
        <DisplayName>Attachment.txt</DisplayName>
        <FieldValues/>
    </Record>
</value>
</KeyValuePair>
...
</FieldValues>
</DynamicRecordItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"
```

```
"https://api.lockpath.app:4443/ComponentService/GetDetailRecord?componentId=10001&recordId=114&embedRichTextImages=true"
```

JSON RESPONSE

```
{
  "Id": 114,
  "DisplayName": 114,
  "FieldValues": [
    ...
    {
      "Key": 34,
      "Value": {
        "Id": 55,
        "DisplayName": "Attachment.txt",
        "FieldValues": []
      }
    }
  ]
}
```

GetDetailRecords

Retrieves details for each record specified. GetDetailRecords also allows multiple sorts to modify the order of the results. The maximum number of records returned (pageSize) is 1000.

Permissions: The authentication account must have Read General Access permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: [https://\[instance name\]:\[port\]/ComponentService/GetDetailRecords](https://[instance name]:[port]/ComponentService/GetDetailRecords)

Method: POST

Input:	componentId (integer):	The Id of the desired component.
	pageIndex (integer):	The index of the page of result to return. Must be ≥ 0 .
	pageSize (integer):	The size of the page results to return. Must be ≥ 1 .
	sortOrder (various) (optional):	The sort parameters the records must meet to be included in the sort order.
	fieldIds (integer) (optional):	The Id of the field to be returned. If not provided, returns all accessible fields. If provided, but empty, returns core system fields (CreatedAt, CreatedBy, etc.). If provided, returns core system fields plus accessible fields. Note that system fields will always be returned regardless.

filters (various) (optional): The filter parameters the records must meet to be counted.

A variety of filters are available, and an unlimited number of search criteria may be applied to each transaction that supports filtering. Search filters are composed of a field path, filter type, and a value.

Element	Description																																																												
FieldPath	Describes the field ID for the column that the records will be filtered on. If the value is stored in the component directly, only one Field Path variable is needed. However, if the value is a lookup to another component, an additional Field Path variable will be required with the column value where the data resides. Field Path variables can be added as many as necessary to provide the correct path to the data.																																																												
FilterType	Describes the ID for the filter being applied. For example, the FilterType for Is Null is 15. If the FilterType would exclude the entry of a value like Is Empty (13) or Is Not Null (16), the <Value> tags should be removed from the request. <table><tr><th colspan="4">Filter Types</th></tr><tr><th>ID</th><th>Option</th><th>ID</th><th>Option</th></tr><tr><td>1</td><td>Contains</td><td>9</td><td>≥</td></tr><tr><td>2</td><td>Excludes</td><td>10</td><td>≤</td></tr><tr><td>3</td><td>Starts With</td><td>11</td><td>Between</td></tr><tr><td>4</td><td>Ends With</td><td>12</td><td>Not Between</td></tr><tr><td>5</td><td>=</td><td>13</td><td>Is Empty</td></tr><tr><td>6</td><td><></td><td>14</td><td>Is Not Empty</td></tr><tr><td>7</td><td>></td><td>15</td><td>Is Null</td></tr><tr><td>8</td><td><</td><td>16</td><td>Is Not Null</td></tr><tr><td>10001</td><td colspan="3">Offset</td></tr><tr><td>10002</td><td colspan="3">Contains Any (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10003</td><td colspan="3">Contains Only (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10004</td><td colspan="3">Contains None (applies only to One-to-Many Lookup fields)</td></tr><tr><td>10005</td><td colspan="3">Contains At Least (applies only to One-to-Many Lookup fields)</td></tr></table>	Filter Types				ID	Option	ID	Option	1	Contains	9	≥	2	Excludes	10	≤	3	Starts With	11	Between	4	Ends With	12	Not Between	5	=	13	Is Empty	6	<>	14	Is Not Empty	7	>	15	Is Null	8	<	16	Is Not Null	10001	Offset			10002	Contains Any (applies only to One-to-Many Lookup fields)			10003	Contains Only (applies only to One-to-Many Lookup fields)			10004	Contains None (applies only to One-to-Many Lookup fields)			10005	Contains At Least (applies only to One-to-Many Lookup fields)		
Filter Types																																																													
ID	Option	ID	Option																																																										
1	Contains	9	≥																																																										
2	Excludes	10	≤																																																										
3	Starts With	11	Between																																																										
4	Ends With	12	Not Between																																																										
5	=	13	Is Empty																																																										
6	<>	14	Is Not Empty																																																										
7	>	15	Is Null																																																										
8	<	16	Is Not Null																																																										
10001	Offset																																																												
10002	Contains Any (applies only to One-to-Many Lookup fields)																																																												
10003	Contains Only (applies only to One-to-Many Lookup fields)																																																												
10004	Contains None (applies only to One-to-Many Lookup fields)																																																												
10005	Contains At Least (applies only to One-to-Many Lookup fields)																																																												
Value	Describes the comparison that the column will be measured against. FilterType 1 applies only to one-to-many lookup fields. For the FilterType 11 and 12, two values are required that are delimited by a pipe () symbol. For the FilterTypes 13 - 16 no values are required.																																																												
SearchCriteriaItem	Describes a single filter. By combining FieldPath, Filter, and Value, a complete search filter is created for the desired parameters.																																																												

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @GetDetailRecords.xml
```

```
"https://api.lockpath.app:4443/ComponentService/GetDetailRecords"
```

XML REQUEST (GetDetailRecords.xml)

```
<GetDetailRecords>
  <componentId>10429</componentId>
  <pageIndex>0</pageIndex>
  <pageSize>2</pageSize>
  <filters>
    <SearchCriteriaItem>
      <FieldPath>
        <int>7455</int>
      </FieldPath>
      <FilterType>3</FilterType>
      <Value>P</Value>
    </SearchCriteriaItem>
  </filters>
  <sortOrder>
    <SortCriteriaItem>
      <FieldPath>
        <int>7457</int>
      </FieldPath>
      <Ascending>true</Ascending>
    </SortCriteriaItem>
  </sortOrder>
  <fieldIds>
    <int>7454</int>
    <int>7455</int>
  </fieldIds>
</GetDetailRecords>
```

XML RESPONSE

```
<ArrayOfDynamicRecordItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <DynamicRecordItem>
    <Id>374</Id>
    <DisplayName>374</DisplayName>
    <FieldValues>
      <KeyValuePair>
        <key>7439</key>
        <value i:type="a:int"
xmlns:a="http://www.w3.org/2001/XMLSchema">374</value>
      </KeyValuePair>
      <KeyValuePair>
        <key>7440</key>
        <value i:type="a:int"
xmlns:a="http://www.w3.org/2001/XMLSchema">3</value>
      </KeyValuePair>
      <KeyValuePair>
        <key>7442</key>
        <value i:type="DynamicRecordItem">
          <Id>12</Id>
          <DisplayName>Name, User</DisplayName>
          <FieldValues/>
        </value>
      </KeyValuePair>
      <KeyValuePair>
        <key>7445</key>
        <value i:type="DynamicRecordItem">
          <Id>12</Id>
          <DisplayName>Name, User</DisplayName>
          <FieldValues/>
        </value>
      </KeyValuePair>
      <KeyValuePair>

```

```

        <key>7448</key>

        <value i:type="a:boolean"
xmlns:a="http://www.w3.org/2001/XMLSchema">false</value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7449</key>

        <value i:type="DynamicRecordItem">

            <Id>301</Id>

            <DisplayName>Published</DisplayName>

            <FieldValues/>

        </value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7451</key>

        <value i:type="a:int"
xmlns:a="http://www.w3.org/2001/XMLSchema">3</value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7454</key>

        <value i:type="a:string"
xmlns:a="http://www.w3.org/2001/XMLSchema">Cable</value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7455</key>

        <value i:type="a:string"
xmlns:a="http://www.w3.org/2001/XMLSchema">Printer</value>

    </KeyValuePair>

</FieldValues>

</DynamicRecordItem>

<DynamicRecordItem>

    <Id>375</Id>

    <DisplayName>375</DisplayName>

    <FieldValues>

        <KeyValuePair>

```

```

        <key>7439</key>

        <value i:type="a:int"
xmlns:a="http://www.w3.org/2001/XMLSchema">375</value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7440</key>

        <value i:type="a:int"
xmlns:a="http://www.w3.org/2001/XMLSchema">1</value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7442</key>

        <value i:type="DynamicRecordItem">

            <Id>12</Id>

            <DisplayName>Name, User</DisplayName>

            <FieldValues/>

        </value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7445</key>

        <value i:type="DynamicRecordItem">

            <Id>12</Id>

            <DisplayName>Name, User</DisplayName>

            <FieldValues/>

        </value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7448</key>

        <value i:type="a:boolean"
xmlns:a="http://www.w3.org/2001/XMLSchema">false</value>

    </KeyValuePair>

    <KeyValuePair>

        <key>7449</key>

        <value i:type="DynamicRecordItem">

```

```

        <Id>301</Id>

        <DisplayName>Published</DisplayName>

        <FieldValues/>
    </value>
</KeyValuePair>
<KeyValuePair>
    <key>7451</key>

    <value i:type="a:int" xmlns:a="http://www.w3.org/2001/XMLSchema">1</value>
</KeyValuePair>
<KeyValuePair>
    <key>7454</key>

    <value i:type="a:string"
xmlns:a="http://www.w3.org/2001/XMLSchema">Keyboard</value>
</KeyValuePair>
<KeyValuePair>
    <key>7455</key>

    <value i:type="a:string"
xmlns:a="http://www.w3.org/2001/XMLSchema">Printer</value>
</KeyValuePair>
</FieldValues>
</DynamicRecordItem>
</ArrayOfDynamicRecordItem>

```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X  
POST -d @GetDetailRecords.json  
"https://api.lockpath.app:4443/ComponentService/GetDetailRecords"
```

JSON REQUEST (GetDetailRecords.json)

```
{  
  "componentId": 10429,  
  "pageIndex": 0,  
  "pageSize": 2,  
  "filters": [{  
    "FieldPath": [7455],  
    "FilterType": 3,  
    "Value": "P"  
  }],  
  "sortOrder": [{  
    "FieldPath": [7457],  
    "Ascending": true  
  }],  
  "fieldIds": [7454, 7455]  
}
```

JSON RESPONSE

```
[
  {
    "Id": 374,
    "DisplayName": "374",
    "FieldValues": [
      {
        "Key": 7439,
        "Value": 374
      },
      {
        "Key": 7440,
        "Value": 3
      },
      {
        "Key": 7442,
        "Value": {
          "Id": 12,
          "DisplayName": "Name, User",
          "FieldValues": []
        }
      },
      {
        "Key": 7445,
        "Value": {
          "Id": 12,
          "DisplayName": "Name, User",
          "FieldValues": []
        }
      },
      {
        "Key": 7448,
```

```

        "Value": false
    },
    {
        "Key": 7449,
        "Value": {
            "Id": 301,
            "DisplayName": "Published",
            "FieldValues": []
        }
    },
    {
        "Key": 7451,
        "Value": 3
    },
    {
        "Key": 7454,
        "Value": "Cable"
    },
    {
        "Key": 7455,
        "Value": "Printer"
    }
]
},
{
    "Id": 375,
    "DisplayName": "375",
    "FieldValues": [
        {
            "Key": 7439,
            "Value": 375
        },
    ],

```

```

{
  "Key": 7440,
  "Value": 1
},
{
  "Key": 7442,
  "Value": {
    "Id": 12,
    "DisplayName": "Name, User",
    "FieldValues": []
  }
},
{
  "Key": 7445,
  "Value": {
    "Id": 12,
    "DisplayName": "Name, User",
    "FieldValues": []
  }
},
{
  "Key": 7448,
  "Value": false
},
{
  "Key": 7449,
  "Value": {
    "Id": 301,
    "DisplayName": "Published",
    "FieldValues": []
  }
},

```

```
[
  {
    "Key": 7451,
    "Value": 1
  },
  {
    "Key": 7454,
    "Value": "Keyboard"
  },
  {
    "Key": 7455,
    "Value": "Printer"
  }
]
```

GetLookupReportColumnFields

Retrieves the field information of each field in a field path that corresponds to a lookup report column. The `lookupFieldId` corresponds to a lookup field with a report definition on it and the `fieldPathId` corresponds to the field path to retrieve fields from, which is obtained from `GetDetailRecord`. `GetLookupReportColumnFields` compliments `GetDetailRecord` by adding additional details about the lookup report columns returned from `GetDetailRecord`.

Permissions: The authentication account must have General Access - Read permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/GetLookupReportColumnFields?lookupFieldId={FIELDID}&fieldPathId={FIELDPATHID}`

Method: GET

Input: `lookupFieldId (integer):` The id of the desired lookup field.
`fieldPathId (integer):` The Id for the desired lookup field path.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetLookupReportColumnFields?lookupFieldId=1234&fieldPathId=5678
```

XML RESPONSE

```
<LookupReportFieldList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Field>
    <Id>3</Id>
    <ComponentId>10001</ComponentId>
    <Name>Workflow Stage</Name>
    <SystemName>WorkflowStage</SystemName>
  </Field>
</LookupReportFieldList>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/ComponentService/GetLookupReportColumnFields?lookupFieldId=  
1234&fieldPathId=5678"
```

JSON RESPONSE

```
[  
  {  
    "Id": 3,  
    "ComponentId": 10001,  
    "Name": "Workflow Stage",  
    "SystemName": "WorkflowStage"  
  }  
]
```

GetRecordAttachment

Retrieves a single attachment for a given field on a record. The record ID and field ID may be found by using GetDetailRecord.

Permissions: The authentication account must have General Access - Read permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/GetRecordAttachment?componentId={COMPONENTID}&recordId={RECORDID}&fieldId={FIELDID}&documentId={DOCUMENTID}`

Method: GET

Input:	componentId (integer):	The Id of the desired component.
	recordId (integer):	The Id for the individual record within the component.
	fieldId (integer):	The Id for the individual field within the component.
	documentId (integer):	The Id for the individual document within the component.

Examples

The Base64 encoded response represents the text in the attachment.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetRecordAttachment?componentId=12345&recordId=1&fieldId=1234&documentId=1
```

XML RESPONSE

```
<RecordAttachmentItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <FileName>Attachment.txt</FileName>
  <FileData>QW4gYXR0YWNobWVudCBmb3IgdGVzdGluZyBwdXJwb3Nlcw==</FileData>
</RecordAttachmentItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
"https://api.lockpath.app:4443/ComponentService/GetRecordAttachment?componentId=12345&recordId=1&fieldId=1234&documentId=1"
```

JSON RESPONSE

```
{  
  "FileName": "Attachment.txt",  
  "FileData": "QW4gYXR0YWNobWVudCBmb3IgdGVzdGluZyBwdXJwb3Nlcw=="  
}
```

GetRecordAttachments

Retrieves a list of information for all attachments associated with the provided component ID, record ID, and field ID. No file data is returned, only file name, field ID, and document ID information.

Permissions: The authentication account must have General Access - Read permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/GetRecordAttachments?componentId={COMPONENTID}&recordId={RECORDID}&fieldId={FIELDID}`

Method: GET

Input:	componentId (integer):	The Id of the desired component.
	recordId (integer):	The Id for the individual record within the component.
	fieldId (integer):	The Id for the individual field within the component.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml; charset=utf-8"
"https://api.lockpath.app:4443/ComponentService/GetRecordAttachments?componentId=10001&recordId=114&fieldId=34"
```

XML RESPONSE

```
<AttachmentInfoList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <AttachmentInfo>
    <FileName>Attachment1.txt</FileName>
    <FieldId>34</FieldId>
    <DocumentId>55</DocumentId>
  </AttachmentInfo>
</AttachmentInfoList>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
"https://api.lockpath.app:4443/ComponentService/GetRecordAttachments?componentId=10001&recordId=114&fieldId=35"
```

JSON RESPONSE

```
[  
  {  
    "FileName": "AttachmentTest.txt",  
    "FieldId": 34,  
    "DocumentId": 55  
  }  
]
```

GetWorkflow

Retrieves workflow details and all workflow stages specified by ID. The ID for a workflow may be found by using GetWorkflows.

Permissions: The authentication account must have:

- Administrative Access - Read permission for the specific component

URL: `https://[instancename]:[port]/ComponentService/GetWorkflow?id=[ID]`

Method: GET

Input: Id (integer): The Id of the desired workflow.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetWorkflow?id=10
```

XML RESPONSE

```
<WorkflowItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>10</Id>
  <Name>Default</Name>
  <IsActive>true</IsActive>
  <IsDefault>true</IsDefault>
  <RoutingCriteria/>
  <WorkflowOwnerGroups/>
  <WorkflowOwnerUsers/>
  <WorkflowStages>
    <WorkflowStage>
      <Id>15</Id>
      <Name>Published</Name>
      <Description/>
      <IsInitial>true</IsInitial>
      <IsActive>true</IsActive>
      <IsVoting>false</IsVoting>
      <GroupAccess>
        <Permission>
```

```
        <Group>
            <Id>0</Id>
            <Name>Everyone</Name>
        </Group>
        <CanViewAll>true</CanViewAll>
        <CanEdit>true</CanEdit>
        <CanTransition>false</CanTransition>
    </Permission>
</GroupAccess>
<UserAccess/>
<UseAssignments>false</UseAssignments>
<Transitions/>
<CanAutoApprove>false</CanAutoApprove>
</WorkflowStage>
</WorkflowStages>
</WorkflowItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/ComponentService/GetWorkflow?id=10
```

JSON RESPONSE

```
{  
  "Id": 10,  
  "Name": "Default",  
  "IsActive": true,  
  "IsDefault": true,  
  "RoutingCriteria": [],  
  "WorkflowOwnerGroups": [],  
  "WorkflowOwnerUsers": [],  
  "WorkflowStages": [  
    {  
      "Id": 15,  
      "Name": "Published",  
      "Description": "",  
      "IsInitial": true,  
      "IsActive": true,  
      "IsVoting": false,  
      "GroupAccess": [  
        {  
          "Group": {  
            "Id": 0,  
            "Name": "Everyone"  
          },  
          "CanViewAll": true,  
          "CanEdit": true,  
          "CanTransition": false  
        }  
      ],  
      "UserAccess": [],  
    }  
  ]  
}
```

```
        "UseAssignments": false,  
        "Transitions": [],  
        "CanAutoApprove": false  
    }  
]  
}
```

GetWorkflows

Retrieves all workflows for a component specified by its Alias. A component is a user-defined data object such as a custom content table. The component Alias may be found by using GetComponentList.

Permissions: The authentication account must have:

- Administrative Access - Read permission for the specific component

URL: `https://[instancename]:[port]/ComponentService/GetWorkflows?componentalias=[ALIAS]`

Method: GET

Input: `componentalias (string):` The Alias of the desired component.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8"
https://api.lockpath.app:4443/ComponentService/GetWorkflows?componentalias=Devices
```

XML RESPONSE

```
<WorkflowList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Workflow>
    <Id>2</Id>
    <Name>Default</Name>
    <IsActive>true</IsActive>
    <IsDefault>true</IsDefault>
  </Workflow>
</WorkflowList>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "Accept: application/json"  
https://api.lockpath.app:4443/ComponentService/GetWorkflows?componentalias=Devices
```

JSON RESPONSE

```
[  
  {  
    "Id": 2,  
    "Name": "Default",  
    "IsActive": true,  
    "IsDefault": true  
  }  
]
```

TransitionRecord

Transitions a record in a workflow stage. The transitionId may be found by using GetWorkflow.

Permissions: The authentication account must have:

- General Access - Read and Update permissions to the defined table
- View and Transition workflow stage permissions
- Record permission

URL: https://[instance name]:[port]/ComponentService/TransitionRecord

Method: POST

Input: tableAlias (string): The Alias for the table.
recordId (integer): The Id of the record to be transitioned.
transitionId (integer): The Id of the workflow stage transition.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @TransitionRecord.xml
```

```
"https://api.lockpath.app:4443/ComponentService/TransitionRecord"
```

XML REQUEST (TransitionRecord.xml)

```
<TransitionRecord>  
  <tableAlias>Devices</tableAlias>  
  <recordId>2</recordId>  
  <transitionId>42</transitionId>  
</TransitionRecord>
```

XML RESPONSE

```
true
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X POST -d @TransitionRecord.json
```

```
"https://api.lockpath.app:4443/ComponentService/TransitionRecord"
```

JSON REQUEST (TransitionRecord.json)

```
{  
  "tableAlias": "Devices",  
  "recordId": "2",  
  "transitionId": "42"  
}
```

JSON RESPONSE

```
true
```

VoteRecord

Casts a vote for a record in a workflow stage. The transition ID may be found by using GetWorkflow.

Permissions: The authentication account must have:

- General Access - Read and Update permissions to the defined table
- View and Vote workflow stage permissions
- Record permission

URL: `https://[instance name]:[port]/ComponentService/VoteRecord`

Method: POST

Input:	tableAlias (string):	The Alias for the table.
	recordId (integer):	The Id of the record to be transitioned.
	transitionId (integer):	The Id of the workflow stage voting rule.
	votingComments (string) (optional):	A comment regarding the vote.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @VoteRecord.xml  
  
"https://api.lockpath.app:4443/ComponentService/VoteRecord"
```

XML REQUEST (VoteRecord.xml)

```
<VoteRecord>  
  <tableAlias>Devices</tableAlias>  
  <recordId>4</recordId>  
  <transitionId>46</transitionId>  
  <votingComments>Approved</votingComments>  
</VoteRecord>
```

XML RESPONSE

```
true
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X POST -d @VoteRecord.json  
  
"https://api.lockpath.app:4443/ComponentService/VoteRecord"
```

JSON REQUEST (VoteRecord.json)

```
{  
  "tableAlias": "Devices",  
  "recordId": "4",  
  "transitionId": "46",  
  "votingComments": "Approved"  
}
```

JSON RESPONSE

```
true
```

CreateRecord

Creates a new record within the specified component of the application.

NOTE: The Required option for a field is only enforced through the user interface, not through the API. Therefore, CreateRecord does not enforce the Required option for fields.

Permissions: The authentication account must have:

- General Access - Create permissions to the selected component
- Edit permission to any field into which data is to be entered

URL: `https://[instance name]:[port]/ComponentService/CreateRecord`

Method: POST

Input: `componentId` (integer): The ID of the desired component.
`dynamicRecord` (FieldValues - KeyValuePair): Fields for the component to be created.

Dynamic Record: A dynamic record is defined by key-value pairs that contain a field ID (where the data will be entered) and a value that contains the type of the data and the data itself.

Applicable types are string, decimal, boolean, dateTime, and DynamicRecordItem. For example, the XML syntax for the types is shown below.

```
<!--Text-->
    <KeyValuePair>
        <key>3664</key>
        <value i:type="a:string">See Spot Run</value>
    </KeyValuePair>

<!--Numeric-->
    <KeyValuePair>
        <key>3667</key>
        <value i:type="a:decimal">25.13</value>
    </KeyValuePair>
```

```

<!--Yes/No-->
  <KeyValuePair>
    <key>3670</key>
    <value i:type="a:boolean">true</value>
  </KeyValuePair>
<!--Date-->
  <KeyValuePair>
    <key>3719</key>
    <value i:type="a:dateTime">2014-09-19T11:02:46</value>
  </KeyValuePair>
<!--1:1 Lookup-->
  <KeyValuePair>
    <key>3667</key>
    <value i:type='DynamicRecordItem'><Id>18</Id></value>
  </KeyValuePair>
<!--1:M Lookup-->
  <KeyValuePair>
    <key>3720</key>
    <value i:type='DynamicRecordList'>
      <Record><Id>13</Id></Record>
      <Record><Id>20</Id></Record>
    </value>
  </KeyValuePair>

```

Empty Fields: If a field is not included in the input for CreateRecord, the field remains empty/null for the created record.

To create empty/null data for a field that is defined in the input is to use:

`<value i:nil="true" />` in XML or `"value i:nil="true"` in JSON

Matrix: Matrix records can only be created after the parent record is created. The Matrix componentId can be retrieved with GetComponentList and the Matrix column fields can be retrieved with GetFieldList, using the Matrix componentId. Matrix row ID's must be retrieved from the table in the database.

Master/Detail: Master/Detail records can only be created after the parent record has been created. The Master/Detail componentId can be retrieved with GetComponentList and the master/detail subfields can be retrieved with GetFieldList, using the Master/Detail componentId.

Workflow Stage Workflow contains a Workflow Stage field ID and the Workflow Stage ID.

ID:

```
<KeyValuePair>
  <key>3849</key>
  <value i:type="DynamicRecordItem"><Id>109</Id></value>
</KeyValuePair>
```

System Fields: The platform populates system fields when a user enters values for their custom created fields. The response contains values for these system fields.

System Fields	
Field	Value Source
CreatedAt	TimeStamp record created
CreatedBy	User Id (API logon)
Id	Unique record identifier
CurrentRevision	Tracks revision history starting at 0
UpdatedAt	TimeStamp record updated
UpdatedBy	User Id (API logon)
Workflow Stage	Workflow stage for record

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d
@CreateRecordInput.xml
"https://api.lockpath.app:4443/ComponentService/CreateRecord"
```

XML REQUEST (CreateRecord.xml)

```
<CreateRecord>
  <componentId>10001</componentId>
  <dynamicRecord xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:a="http://www.w3.org/2001/XMLSchema">
    <FieldValues>
      <KeyValuePair>
        <key>6</key>
        <value i:type="a:string">Tag</value>
      </KeyValuePair>
    </FieldValues>
  </dynamicRecord>
</CreateRecord>
```

XML RESPONSE

```
<DynamicRecordItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>321</Id>
  <DisplayName>321</DisplayName>
  <FieldValues>
    <KeyValuePair>
      <key>3</key>
      <value i:nil="true"/>
    </KeyValuePair>
    <KeyValuePair>
      <key>4</key>
      <value i:nil="true"/>
    </KeyValuePair>
    <KeyValuePair>
      <key>6</key>
```

```

        <value i:type="a:string"
xmlns:a="http://www.w3.org/2001/XMLSchema">Tag</value>

    </KeyValuePair>

...

</FieldValues>

</DynamicRecordItem>

```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @CreateRecord.json
```

```
"https://api.lockpath.app:4443/ComponentService/CreateRecord"
```

JSON REQUEST (CreateRecord.json)

```

{
    "componentId": "10001",
    "dynamicRecord": {
        "FieldValues": [
            {
                "key": "3",
                "value": "API Example DNS Name"
            },
            {
                "key": "9",
                "value": 250.00
            },
            {
                "key": "10",
                "value": "12/25/2020"
            },
            {
                "key": "11",
                "value": "127.0.0.1"
            }
        ]
    }
}

```

```
}
```

JSON RESPONSE

```
{
  "Id": 322,
  "DisplayName": "322",
  "FieldValues": [
    {
      "Key": 3,
      "Value": "API Example DNS Name"
    },
    {
      "Key": 4,
      "Value": null
    },
    {
      "Key": 6,
      "Value": null
    },
    {
      "Key": 7,
      "Value": null
    },
    {
      "Key": 8,
      "Value": null
    },
    {
      "Key": 9,
      "Value": 250.0
    },
    {
      "Key": 10,
```

```
        "Value": "2020-12-25T00:00:00"
      },
      {
        "Key": 11,
        "Value": "127.0.0.1"
      },
      ...
    ]
  }
}
```

UpdateRecord

Updates a record within the specified component of the application.

NOTE: The Required option for a field is only enforced through the user interface, not through the API. Therefore, UpdateRecord does not enforce the Required option for fields. The response will include the complete set of fields for the specified record.

Permissions: The authentication account must have:

- Read/Update General Access permission to the selected component
- Edit permission for fields to be updated
- Read permission to the selected record

URL: [https://\[instance name\]:\[port\]/ComponentService/UpdateRecord](https://[instance name]:[port]/ComponentService/UpdateRecord)

Method: POST

Input: componentId (integer): The ID of the desired component.
dynamicRecord (FieldValues - KeyValuePair): Fields for the component to be created.

Dynamic Record: A dynamic record is defined by key-value pairs that contain a field ID (where the data will be entered) and a value that contains the type of the data and the data itself.

Applicable types are string, decimal, boolean, dateTime, and DynamicRecordItem. For example, the XML syntax for the types is shown below.

```
<!--Text-->
    <KeyValuePair>
        <key>3664</key>
        <value i:type="a:string">See Spot Run</value>
    </KeyValuePair>
<!--Numeric-->
    <KeyValuePair>
        <key>3667</key>
        <value i:type="a:decimal">25.13</value>
    </KeyValuePair>
<!--Date-->
    <KeyValuePair>
        <key>3719</key>
        <value i:type="a:dateTime">2014-09-19T11:02:46</value>
    </KeyValuePair>
```

```

<!--Yes/No-->
    <KeyValuePair>
        <key>3670</key>
        <value i:type="a:boolean">true</value>
    </KeyValuePair>
<!--1:1 Lookup-->
    <KeyValuePair>
        <key>3667</key>
        <value i:type='DynamicRecordItem'><Id>18</Id></value>
    </KeyValuePair>
<!--1:M Lookup-->
    <KeyValuePair>
        <key>3720</key>
        <value i:type='DynamicRecordList'>
            <Record><Id>13</Id></Record>
            <Record><Id>20</Id></Record>
        </value>
    </KeyValuePair>

```

Empty Fields: To create empty/null data for a field that is defined in the input is to use:

`<value i:nil="true" />` in XML or `"value i:nil="true"` in JSON

Matrix: The Matrix componentId can be retrieved with GetComponentList and the Matrix column fields can be retrieved with GetFieldList, using the Matrix componentId. Matrix row ID's must be retrieved from the table in the database.

Master/Detail: The Master/Detail componentId can be retrieved with GetComponentList and the master/detail subfields can be retrieved with GetFieldList, using the Master/Detail componentId.

Examples

In the samples, an ellipse (...) indicates an incomplete list of items returned.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d
@UpdateRecordInput.xml
"https://api.lockpath.app:4443/ComponentService/UpdateRecord"
```

XML REQUEST (UpdateRecord.xml)

```
<UpdateRecord>
  <componentId>10001</componentId>
  <dynamicRecord xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns:a="http://www.w3.org/2001/XMLSchema">
    <Id>1</Id>
    <FieldValues>
      <KeyValuePair>
        <key>9</key>
        <value i:type="a:decimal">120.00</value>
      </KeyValuePair>
    </FieldValues>
  </dynamicRecord>
</UpdateRecord>
```

XML RESPONSE

```
<DynamicRecordItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <Id>1</Id>
  <DisplayName>1</DisplayName>
  <FieldValues>
    <KeyValuePair>
      <key>9</key>
      <value i:type="a:decimal"
xmlns:a="http://www.w3.org/2001/XMLSchema">120.00</value>
    </KeyValuePair>
    ...
  </FieldValues>
</DynamicRecordItem>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @UpdateRecord.json

"https://api.lockpath.app:4443/ComponentService/UpdateRecord"
```

JSON REQUEST (UpdateRecord.json)

```
{
  "componentId": "10001",
  "dynamicRecord": {
    "Id": "314",
    "FieldValues": {
      "key": "6",
      "value": "Asset: IT Department"
    }
  }
}
```

JSON RESPONSE

```
{
  "Id": "314",
  "DisplayName": "314",
  "FieldValues": [
    {
      "Key": "6",
      "Value": "Asset: IT Department"
    },
    ...
  ]
}
```

UpdateRecordAttachments

Adds new attachments and/or updates existing attachments to the provided documents field(s) on a specific record, where the FileData is represented as a Base64 string. The maximum data size of the request is controlled by the `maxAllowedContentLength` and `maxReceivedMessageSize` values in the API `web.config` file.

Permissions: The authentication account must have Read and Update General Access permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/UpdateRecordAttachments`

Method: POST

Input: `componentId` (integer): The Id of the desired component.
`recordId` (integer): The Id for the individual record within the component.
`fieldId` (integer): The Id for the individual field within the component.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d
@UpdateRecordAttachments.xml

"https://api.lockpath.app:4443/ComponentService/UpdateRecordAttachments"
```

XML REQUEST (UpdateRecordAttachments.xml)

```
<UpdateRecordAttachments>

<componentId>10001</componentId>

<dynamicRecord xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns:a="http://www.w3.org/2001/XMLSchema">

<Id>2</Id>

<FieldValues>

  <KeyValuePair>

    <key>34</key>

    <value i:type='RecordAttachmentList'>

      <Attachment>

        <FileName>AttachmentTest1.txt</FileName>

        <FileData>SGVsbG8gV29ybGQ=</FileData>

      </Attachment>

    </value>

  </KeyValuePair>

</FieldValues>

</dynamicRecord>

</UpdateRecordAttachments>
```

```

        </value>
    </KeyValuePair>
</FieldValues>
</dynamicRecord>
</UpdateRecordAttachments>

```

XML RESPONSE

```

<AttachmentOperationResultList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
    <AttachmentOperationResult>
        <OperationSucceeded>true</OperationSucceeded>
        <Message>Attachment was successfully added to the Documents field.</Message>
        <ComponentId>10001</ComponentId>
        <RecordId>2</RecordId>
        <AttachmentInfo>
            <FileName>AttachmentTest1.txt</FileName>
            <FieldId>34</FieldId>
            <DocumentId>56</DocumentId>
        </AttachmentInfo>
    </AttachmentOperationResult>
</AttachmentOperationResultList>

```

JSON REQUEST (cURL)

```

curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @UpdateRecordAttachments.json
"https://api.lockpath.app:4443/ComponentService/UpdateRecordAttachments"

```

JSON REQUEST (UpdateRecordAttachments.json)

```

{
    "componentId": 10001,
    "dynamicRecord": {
        "Id": 2,
        "FieldValues": [
            {
                "key": 34,
                "value":

```

```
    [
      {
        "FileName": "AttachmentTest1.txt",
        "FileData": "SGVsbg8gV29ybGQ="
      }
    ]
  }
}
```

JSON RESPONSE

```
[
  {
    "OperationSucceeded": true,
    "Message": "Attachment was successfully updated on the Documents field.",
    "ComponentId": 10001,
    "RecordId": 2,
    "AttachmentInfo": {
      "FileName": "AttachmentTest1.txt",
      "FieldId": 34,
      "DocumentId": 56
    }
  }
]
```

DeleteRecord

Deletes a selected record from within a chosen component. DeleteRecord will update the record, making it so it will no longer be viewable.

IMPORTANT: Records are soft deleted to maintain any historical references to the record and can be restored with a database script.

Permissions: The authentication account must have:

- General Access - Read and Delete permissions to the selected component
- Read permissions to the selected record

URL: `https://[instance name]:[port]/ComponentService/DeleteRecord`

Method: DELETE

Input: `componentId (integer):` The Id of the desired component.
`recordId (integer):` The Id for the individual record within the component.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X DELETE -d
@DeleteRecord.xml

"https://api.lockpath.app:4443/ComponentService/DeleteRecord"
```

XML REQUEST (DeleteRecord.xml)

```
<DeleteRecord>
  <componentId>10001</componentId>
  <recordId>319</recordId>
</DeleteRecord>
```

XML RESPONSE

```
<boolean xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</boolean>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
DELETE-d @DeleteRecord.json

"https://api.lockpath.app:4443/ComponentService/DeleteRecord"
```

JSON REQUEST (DeleteRecord.json)

```
{  
  "componentId": "10001",  
  "recordId": "319"  
}
```

JSON RESPONSE

```
true
```

DeleteRecordAttachments

Deletes the specified attachments from the provided document fields on a specific record.

Permissions: The authentication account must have General Access - Read and Delete permissions to:

- Selected component
- Selected record
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ComponentService/DeleteRecordAttachments`

Method: POST

Input:

<code>componentId (integer):</code>	The Id of the desired component.
<code>recordId (integer):</code>	The Id for the individual record within the component.
<code>fieldId (integer):</code>	The Id for the individual field within the component.
<code>attachmentId (integer):</code>	The Id for the individual attachment within the component.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @DeleteRecordAttachments.xml
```

```
"https://api.lockpath.app:4443/ComponentService/DeleteRecordAttachments"
```

XML REQUEST (DeleteRecordAttachments.xml)

```
<DeleteRecordAttachments>

<componentId>10001</componentId>

<dynamicRecord xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns:a="http://www.w3.org/2001/XMLSchema">

  <Id>1</Id>

  <FieldValues>

    <KeyValuePair>

      <key>34</key>

      <value i:type='DynamicRecordList'>

        <Record>

          <Id>1</Id>

        </Record>

      </value>

    </KeyValuePair>

  </FieldValues>

</dynamicRecord>

</DeleteRecordAttachments>
```

```

        </KeyValuePair>
    </FieldValues>
</dynamicRecord>
</DeleteRecordAttachments>

```

XML RESPONSE

```

<AttachmentOperationResultList xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
    <AttachmentOperationResult>
        <OperationSucceeded>true</OperationSucceeded>
        <Message>Attachment was successfully deleted from the Documents
field.</Message>
        <ComponentId>10001</ComponentId>
        <RecordId>1</RecordId>
        <AttachmentInfo>
            <FileName/>
            <FieldId>34</FieldId>
            <DocumentId>1</DocumentId>
        </AttachmentInfo>
    </AttachmentOperationResult>
</AttachmentOperationResultList>

```

JSON REQUEST (cURL)

```

curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @DeleteRecordAttachments.json

"https://api.lockpath.app:4443/ComponentService/DeleteRecordAttachments"

```

JSON REQUEST (DeleteRecordAttachments.json)

```

{
    "componentId": "10001",
    "dynamicRecord": {
        "Id": "2",
        "FieldValues": [
            {
                "key": "34",
                "value": [

```

```
        {
          "Id": "1"
        }
      ]
    }
  ]
}
```

JSON RESPONSE

```
[
  {
    "OperationSucceeded": true,
    "Message": "Attachment was successfully deleted from the Documents field.",
    "ComponentId": 10001,
    "RecordId": 2,
    "AttachmentInfo": {
      "FileName": "Attachment.txt",
      "FieldId": 34,
      "DocumentId": 1
    }
  }
]
```

ImportFile

Queues a job to import a file for a defined import template.

Permissions: The authentication account must have:

- General Access - Read, Create, Update, and Import/Bulk permissions to the defined table

To enable the Run As System option the authentication account must have also have Administrative Access - Read, Create, and Update permissions to the defined table

URL: `https://[instance name]:[port]/ComponentService/ImportFile`

Method: POST

Input:	<code>tableAlias (string):</code>	The Alias for the table to import into.
	<code>importTemplateName (string):</code>	The Name of the import template.
	<code>fileName (string):</code>	The Name of the import file.
	<code>fileData (string):</code>	Base64 encoded string of file contents.
	<code>runAsSystem (boolean):</code>	Run import as the System account rather than authentication account.

Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @ImportFile.xml
```

```
"https://api.lockpath.app:4443/ComponentService/ImportFile"
```

XML REQUEST (ImportFile.xml)

```
<ImportFile>
  <tableAlias>_temp</tableAlias>
  <importTemplateName>CSV Import</importTemplateName>
  <fileName>import_temp.csv</fileName>
  <fileData>SGVsbG8gV29ybGQ=</fileData>
  <runAsSystem>false</runAsSystem>
</ImportFile>
```

XML RESPONSE

```
<boolean xmlns="http://schemas.microsoft.com/2003/10/Serialization/">true</boolean>
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X
POST -d @ImportFile.json

"https://api.lockpath.app:4443/ComponentService/ImportFile"
```

JSON REQUEST (ImportFile.json)

```
{
  "tableAlias": "_temp",
  "importTemplateName": "CSV Import",
  "fileName": "import_temp.csv",
  "fileData": "SGVsbG8gV29ybGQ=",
  "runAsSystem": "false"
}
```

JSON RESPONSE

```
true
```

4: Report Services API

This chapter includes a description of the available methods for reporting about the data in the NAVEX IRM Platform.

ExportReport	151
--------------------	-----

ExportReport

Outputs report data to a CSV, PDF, or XLSX file format. The My Reports tab in the user interface contains the ID for the desired report. If a report is not found or permissions are not granted to export data, the message "Report does not exist or is inaccessible" appears.

Permissions: The authentication account must have General Access - Read and Print/Export permissions to:

- Applicable component
- Applicable records
- Applicable fields in the component (table)

URL: `https://[instance name]:[port]/ReportService/ExportReport`

Method: GET

Input: Id (integer): The Id of the desired report.

fileExtension (text): CSV (a comma-separated values file).

PDF (a portable document format file).

XLSX (a Microsoft Excel Open XML formatted spreadsheet).

Examples

The resource media type for this method is application/octet-stream. The XML and JSON responses include the exported data in the format you request.

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -H "Accept: application/xml" https://api.lockpath.app:4443/ReportService/ExportReport?id=123&fileExtension=csv
```

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" https://api.lockpath.app:4443/ReportService/ExportReport?id=123&fileExtension=csv
```

5: Assessment Services API

This chapter includes the available methods for initiating assessments into fields on DCF tables and on Master Detail records in the NAVEX IRM Platform.

IssueAssessment	153
-----------------------	-----

IssueAssessment

Issues an assessment according to a specified schedule. Assessments can be issued directly to a vendor or you can issue an assessment internally to a NAVEX IRM user to perform assessments of a vendor without the assessment being viewable by external vendor contacts that have access to the NAVEX IRM Vendor Portal.

Permissions: The authentication account must have General Access - Create, Read and Update permissions to:

- Applicable component
- Applicable records
- Applicable fields in the component (table)

The user account must have "Accessible to" permissions to:

- Applicable assessment category

URL: [https://\[instance name\]:\[port\]/AssessmentService/IssueAssessment](https://[instance name]:[port]/AssessmentService/IssueAssessment)

Method: POST

Input: Various user fields

Schedule Types

Assessments are generated according to a schedule type and are issued on a cycle of every five minutes.

Schedule Type	Description
Immediate	A one-time assessment issued today.
Onetime	A one-time assessment issued at a future date.
Recurring	An assessment that repeats how often you want it to be issued according to a schedule.

Parameters

The following is a list of the available parameters for the IssueAssessment method. Some parameters when enabled require additional parameters to be set.

Parameter	Additional parameters	Description
ProjectId		Specifies the id of the project defined in Compliance Manager associated with the assessment. When an assessment is issued from a DCF table or master/detail record this parameter is not required.
TableId		Specifies the id of the table to which the assessment is associated. For master/detail record assessments, TableId is the master field ID.
FieldId		Specifies the id of the field to which the assessment is associated. For master/detail record assessments, FieldId is the assessment field ID of the master/detail table.
ContentId		Specifies the id of the record of the table to which the assessment is associated. For master/detail record assessments, ContentId is the specific record within the master/detail table.
VendorId		Specifies the id of the vendor for the issuance. When an assessment is issued to a vendor this parameter is required.
IsVendorInternalMode		Although this parameter exists, it has no content.
Name		Specifies the name identifying the assessment.
TemplateId		Specifies the id of the template containing the questions for the assessment.
VendorContactId		Specifies the id of the vendor contact. When an assessment is issued to a vendor this parameter is required.

Parameter	Additional parameters	Description
UsersIds		Specifies the id of the user to which you want to issue the assessment. Use a comma to separate the ids of multiple users.
GroupIds		Specifies the id of the group to which you want to issue the assessment. Use a comma to separate the ids of multiple groups.
AssignedUserOnly		Determines whether a review or multiple reviewers are required. When this parameter is set to True , optional parameters are required.
	ReviewerId	Specifies the id of the user designated to review the assessment once it has been submitted. Use a comma to separate the ids of multiple reviewers.
AllowDelegation		Determines whether to allow the assessment to be delegated.
GenerateFindings		Determines whether to automatically generate findings.
	FindingsOutputTableId	Specifies the table (ComponentId) into which you want the record automatically generated for the question specified in the applicable assessment template. This parameter appears when GenerateFindings is set to True.
	FindingsOutputFieldId	Specifies the field (FieldId) into which you want the record automatically generated for the question specified in the applicable assessment template. This parameter appears when GenerateFindings is set to True.
	FindingsParentGroupingFieldId	Specifies the field (FieldId) into which the findings are grouped together and recorded in one single table record. This parameter appears when GenerateFindings is set to True.

Parameter	Additional parameters	Description
	FindingsControlsFieldId	Specifies the field (FieldId) into which the controls that are mapped to the assessment question are automatically populated. This parameter appears when GenerateFindings is set to True.
ShowUserScores		Determines whether user scores are written to a record. This parameter is only applicable if the assessment template has score-based enabled.
IssuanceScheduleType		Specifies whether to issue the assessment immediately, onetime, or on a recurring basis.
	IssueDate	Specifies the issuance date of the assessment. The default is today's date. This parameter appears when IssuanceScheduleType is set to onetime.
	DueDate	Specifies the due date of the issuance. The date may not precede the issuance date. This parameter appears when IssuanceScheduleType is set to onetime.
	BeginningDate	Specifies the date when the assessment is initially issued. This parameter appears when IssuanceScheduleType is set to recurring.
	EndingDate	Specifies the date when the issuing of the assessment is to stop. This parameter appears when IssuanceScheduleType is set to recurring.
	RepeatInterval	Specifies whether the issuance repeats Daily, Weekly, Monthly, or Annually. This parameter appears when IssuanceScheduleType is set to recurring.

Parameter	Additional parameters	Description
	RepeatUnit	Specifies the number of times the assessment is repeated. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Daily, Monthly, or Annually.
	RepeatsSunday	Determines whether the issuance repeats on Sunday. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Weekly.
	RepeatsMonday	Determines whether the issuance repeats on Monday. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Weekly.
	RepeatsTuesday	Determines whether the issuance repeats on Tuesday. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Weekly.
	RepeatsWednesday	Determines whether the issuance repeats on Wednesday. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Weekly.
	RepeatsThursday	Determines whether the issuance repeats on Thursday. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Weekly.
	RepeatsFriday	Determines whether the issuance repeats on Friday. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Weekly.

Parameter	Additional parameters	Description
	RepeatsSaturday	Determines whether the issuance repeats on Saturday. This parameter appears when IssuanceScheduleType is set to recurring and is required when RepeatInterval is set to Weekly.
	DueUnit	Specifies the number of days, weeks, months, or years until the assessment is due. This parameter appears when IssuanceScheduleType is set to recurring.
	DueInterval	Determines whether the due date interval is days, weeks, months, or years. This parameter appears when IssuanceScheduleType is set to recurring.
PrepopulatePriorAnswers		Determines whether the questions specified in the applicable assessment template are populated with the answers submitted previously.
EmailSubject		Specifies the text in the subject of the email that is sent notifying the issuer of the assessment.
EmailBody		Specifies the text in the body of the email that is sent notifying the issuer of the assessment.
SendCategoryEmail		Determines whether email is sent to the assessment category owner.
SendReviewersOrIssueEmail		Determines whether email is sent to reviewers.

Parameter	Additional parameters	Description
	AdministrativeEmailSubject	Specifies the text in the subject of the email that is sent to the user designated to review the assessment. This parameter appears when SendReviewersOrIssueEmail is set to True.
	AdministrativeEmailBody	Specifies the text in the body of the email that is sent to the user designated to review the assessment. This parameter appears when SendReviewersOrIssueEmail is set to True.
ScheduledAssessmentId		Specifies the Id of the assessment itself and is found at the end of the response object.

XML Examples

XML REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/xml;charset=utf-8" -X POST -d @Request.xml  
  
"https://api.lockpath.app:4443/AssessmentService/IssueAssessment"
```

XML REQUEST - Immediate Assessment (Results.xml)

```
<IssueAssessment>  
  
  <assessmentIssuance xmlns:i="http://www.w3.org/2001/XMLSchema-instance"  
xmlns:a="http://www.w3.org/2001/XMLSchema">  
  
    <ProjectId>16</ProjectId>  
  
    <Name>Assessment Issued Today</Name>  
  
    <TemplateId>21</TemplateId>  
  
    <UserIds>1027</UserIds>  
  
    <GroupIds></GroupIds>  
  
    <AllowDelegation>false</AllowDelegation>  
  
    <GenerateFindings>false</GenerateFindings>  
  
    <AssignedUserOnly>false</AssignedUserOnly>  
  
    <ShowUserScores>false</ShowUserScores>  
  
    <IssuanceScheduleType>immediate</IssuanceScheduleType>  
  
    <DueDate>3/12/2021</DueDate>  
  
    <PrepopulatePriorAnswers>false</PrepopulatePriorAnswers>  
  
    <EmailSubject>Immediate Assessment</EmailSubject>  
  
    <EmailBody>Immediate Assessment</EmailBody>  
  
  </assessmentIssuance>  
  
</IssueAssessment>
```

XML RESPONSE - Immediate Assessment

```
<AssessmentIssuanceItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">  
  
  <ProjectId>16</ProjectId>  
  
  <TableId i:nil="true"/>  
  
  <FieldId i:nil="true"/>  
  
  <ContentId i:nil="true"/>  
  
  <VendorId i:nil="true"/>  
  
  <IsVendorInternalMode i:nil="true"/>  
</AssessmentIssuanceItem>
```

```

<Name>Assessment Issued Today</Name>
<TemplateId>21</TemplateId>
<UserIds>1027</UserIds>
<VendorContactId i:nil="true"/>
<GroupIds/>
<AllowDelegation>false</AllowDelegation>
<AssignedUserOnly i:nil="true"/>
<ReviewerId i:nil="true"/>
<GenerateFindings>false</GenerateFindings>
<FindingsOutputTableId i:nil="true"/>
<FindingsOutputFieldId i:nil="true"/>
<FindingsParentGroupingFieldId i:nil="true"/>
<FindingsControlsFieldId i:nil="true"/>
<ShowUserScores>false</ShowUserScores>
<IssuanceScheduleType>immediate</IssuanceScheduleType>
<BeginningDate i:nil="true"/>
<IssueDate i:nil="true"/>
<DueDate>3/12/2021</DueDate>
<EndingDate i:nil="true"/>
<RepeatUnit i:nil="true"/>
<RepeatInterval i:nil="true"/>
<RepeatsSunday i:nil="true"/>
<RepeatsMonday i:nil="true"/>
<RepeatsTuesday i:nil="true"/>
<RepeatsWednesday i:nil="true"/>
<RepeatsThursday i:nil="true"/>
<RepeatsFriday i:nil="true"/>
<RepeatsSaturday i:nil="true"/>
<DueUnit i:nil="true"/>
<DueInterval i:nil="true"/>
<PrepopulatePriorAnswers>false</PrepopulatePriorAnswers>
<EmailSubject>Immediate Assessment</EmailSubject>

```

```

    <EmailBody>Immediate Assessment</EmailBody>

    <SendReviewerOrIssuerEmail i:nil="true"/>

    <SendCategoryEmail i:nil="true"/>

    <AdministrativeEmailSubject i:nil="true"/>

    <AdministrativeEmailBody i:nil="true"/>

    <ScheduledAssessmentId>165</ScheduledAssessmentId>

</AssessmentIssuanceItem>

```

XML REQUEST - Onetime Scheduled Assessment (Results.xml)

```

<IssueAssessment>

    <assessmentIssuance xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns:a="http://www.w3.org/2001/XMLSchema">

        <ProjectId>16</ProjectId>

        <Name>Onetime Scheduled Assessment</Name>

        <TemplateId>21</TemplateId>

        <UserIds>1027</UserIds>

        <GroupIds></GroupIds>

        <AllowDelegation>false</AllowDelegation>

        <GenerateFindings>false</GenerateFindings>

        <AssignedUserOnly>false</AssignedUserOnly>

        <ShowUserScores>false</ShowUserScores>

        <IssuanceScheduleType>onetime</IssuanceScheduleType>

        <IssueDate>03/05/2021</IssueDate>

        <DueDate>03/31/2021</DueDate>

        <PrepopulatePriorAnswers>false</PrepopulatePriorAnswers>

        <EmailSubject>Onetime Scheduled Assessment</EmailSubject>

        <EmailBody>Onetime Scheduled Assessment</EmailBody>

    </assessmentIssuance>

</IssueAssessment>

```

XML RESPONSE - Onetime Scheduled Assessment

```

<AssessmentIssuanceItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">

    <ProjectId>16</ProjectId>

    <TableId i:nil="true"/>

    <FieldId i:nil="true"/>

```

```
<ContentId i:nil="true"/>
<VendorId i:nil="true"/>
<IsVendorInternalMode i:nil="true"/>
<Name>Onetime Scheduled Assessment</Name>
<TemplateId>21</TemplateId>
<UserIds>1027</UserIds>
<VendorContactId i:nil="true"/>
<GroupIds/>
<AllowDelegation>false</AllowDelegation>
<AssignedUserOnly i:nil="true"/>
<ReviewerId i:nil="true"/>
<GenerateFindings>false</GenerateFindings>
<FindingsOutputTableId i:nil="true"/>
<FindingsOutputFieldId i:nil="true"/>
<FindingsParentGroupingFieldId i:nil="true"/>
<FindingsControlsFieldId i:nil="true"/>
<ShowUserScores>false</ShowUserScores>
<IssuanceScheduleType>onetime</IssuanceScheduleType>
<BeginningDate i:nil="true"/>
<IssueDate>03/05/2021</IssueDate>
<DueDate>03/31/2021</DueDate>
<EndingDate i:nil="true"/>
<RepeatUnit i:nil="true"/>
<RepeatInterval i:nil="true"/>
<RepeatsSunday i:nil="true"/>
<RepeatsMonday i:nil="true"/>
<RepeatsTuesday i:nil="true"/>
<RepeatsWednesday i:nil="true"/>
<RepeatsThursday i:nil="true"/>
<RepeatsFriday i:nil="true"/>
<RepeatsSaturday i:nil="true"/>
<DueUnit i:nil="true"/>
```

```

    <DueInterval i:nil="true"/>

    <PrepopulatePriorAnswers>false</PrepopulatePriorAnswers>

    <EmailSubject>Onetime Scheduled Assessment</EmailSubject>

    <EmailBody>Onetime Scheduled Assessment</EmailBody>

    <SendReviewerOrIssuerEmail i:nil="true"/>

    <SendCategoryEmail i:nil="true"/>

    <AdministrativeEmailSubject i:nil="true"/>

    <AdministrativeEmailBody i:nil="true"/>

    <ScheduledAssessmentId>168</ScheduledAssessmentId>

</AssessmentIssuanceItem>

```

XML REQUEST - Recurring Scheduled Assessment (Results.xml)

```

<IssueAssessment>

    <assessmentIssuance xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns:a="http://www.w3.org/2001/XMLSchema">

        <TableId>10383</TableId>

        <FieldId>6543</FieldId>

        <ContentId>1</ContentId>

        <Name>Recurring Scheduled Assessment</Name>

        <TemplateId>21</TemplateId>

        <UserIds>1027</UserIds>

        <GroupIds></GroupIds>

        <AllowDelegation>false</AllowDelegation>

        <GenerateFindings>false</GenerateFindings>

        <AssignedUserOnly>false</AssignedUserOnly>

        <ShowUserScores>false</ShowUserScores>

        <IssuanceScheduleType>recurring</IssuanceScheduleType>

        <BeginningDate>03/03/2021</BeginningDate>

        <RepeatUnit>monthly</RepeatUnit>

        <EndingDate>11/05/2021</EndingDate>

        <RepeatInterval>1</RepeatInterval>

        <DueUnit>daily</DueUnit>

        <DueInterval>1</DueInterval>

        <EmailSubject>Recurring Scheduled Assessment</EmailSubject>
    </assessmentIssuance>
</IssueAssessment>

```

```

    <EmailBody>Recurring Scheduled Assessment</EmailBody>

  </assessmentIssuance>

</IssueAssessment>

```

XML RESPONSE - Recurring Scheduled Assessment

```

<AssessmentIssuanceItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">

  <ProjectId i:nil="true"/>

  <TableId>10383</TableId>

  <FieldId>6543</FieldId>

  <ContentId>1</ContentId>

  <VendorId i:nil="true"/>

  <IsVendorInternalMode i:nil="true"/>

  <Name>Recurring Scheduled Assessment</Name>

  <TemplateId>21</TemplateId>

  <UserIds>1027</UserIds>

  <VendorContactId i:nil="true"/>

  <GroupIds/>

  <AllowDelegation>false</AllowDelegation>

  <AssignedUserOnly i:nil="true"/>

  <ReviewerId i:nil="true"/>

  <GenerateFindings>false</GenerateFindings>

  <FindingsOutputTableId i:nil="true"/>

  <FindingsOutputFieldId i:nil="true"/>

  <FindingsParentGroupingFieldId i:nil="true"/>

  <FindingsControlsFieldId i:nil="true"/>

  <ShowUserScores>false</ShowUserScores>

  <IssuanceScheduleType>recurring</IssuanceScheduleType>

  <BeginningDate>03/03/2021</BeginningDate>

  <IssueDate i:nil="true"/>

  <DueDate i:nil="true"/>

  <EndingDate i:nil="true"/>

  <RepeatUnit>monthly</RepeatUnit>

  <RepeatInterval>1</RepeatInterval>

  <RepeatsSunday i:nil="true"/>

```

```

<RepeatsMonday i:nil="true"/>
<RepeatsTuesday i:nil="true"/>
<RepeatsWednesday i:nil="true"/>
<RepeatsThursday i:nil="true"/>
<RepeatsFriday i:nil="true"/>
<RepeatsSaturday i:nil="true"/>
<DueUnit>daily</DueUnit>
<DueInterval>1</DueInterval>
<PrepopulatePriorAnswers i:nil="true"/>
<EmailSubject>Recurring Scheduled Assessment</EmailSubject>
<EmailBody>Recurring Scheduled Assessment</EmailBody>
<SendReviewerOrIssuerEmail i:nil="true"/>
<SendCategoryEmail i:nil="true"/>
<AdministrativeEmailSubject i:nil="true"/>
<AdministrativeEmailBody i:nil="true"/>
<ScheduledAssessmentId>169</ScheduledAssessmentId>
</AssessmentIssuanceItem>

```

XML REQUEST - Vendor Internal Assessment (Results.xml)

```

<IssueAssessment>
  <assessmentIssuance xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns:a="http://www.w3.org/2001/XMLSchema">
    <TableId>10066</TableId>
    <FieldId>1439</FieldId>
    <ContentId>7</ContentId>
    <VendorId>7</VendorId>
    <Name>Vendor Internal Assessment</Name>
    <TemplateId>21</TemplateId>
    <VendorContactId>1048</VendorContactId>
    <AllowDelegation>false</AllowDelegation>
    <GenerateFindings>false</GenerateFindings>
    <AssignedUserOnly>false</AssignedUserOnly>
    <ShowUserScores>false</ShowUserScores>
    <IssuanceScheduleType>immediate</IssuanceScheduleType>
  </assessmentIssuance>
</IssueAssessment>

```

```

    <DueDate>03/12/2021</DueDate>

    <PrepopulatePriorAnswers>false</PrepopulatePriorAnswers>

    <EmailSubject>Vendor Internal Assessment</EmailSubject>

    <EmailBody>Vendor Internal Assessment</EmailBody>

  </assessmentIssuance>

</IssueAssessment>

```

XML RESPONSE - Vendor Internal Assessment

```

<AssessmentIssuanceItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
  <ProjectId i:nil="true"/>
  <TableId>10066</TableId>
  <FieldId>1439</FieldId>
  <ContentId>7</ContentId>
  <VendorId>7</VendorId>
  <IsVendorInternalMode i:nil="true"/>
  <Name>Vendor Internal Assessment</Name>
  <TemplateId>21</TemplateId>
  <UserIds i:nil="true"/>
  <VendorContactId>1048</VendorContactId>
  <GroupIds i:nil="true"/>
  <AllowDelegation>false</AllowDelegation>
  <AssignedUserOnly i:nil="true"/>
  <ReviewerId i:nil="true"/>
  <GenerateFindings>false</GenerateFindings>
  <FindingsOutputTableId i:nil="true"/>
  <FindingsOutputFieldId i:nil="true"/>
  <FindingsParentGroupingFieldId i:nil="true"/>
  <FindingsControlsFieldId i:nil="true"/>
  <ShowUserScores>false</ShowUserScores>
  <IssuanceScheduleType>immediate</IssuanceScheduleType>
  <BeginningDate i:nil="true"/>
  <IssueDate i:nil="true"/>
  <DueDate>03/12/2021</DueDate>
  <EndingDate i:nil="true"/>

```

```

<RepeatUnit i:nil="true"/>
<RepeatInterval i:nil="true"/>
<RepeatsSunday i:nil="true"/>
<RepeatsMonday i:nil="true"/>
<RepeatsTuesday i:nil="true"/>
<RepeatsWednesday i:nil="true"/>
<RepeatsThursday i:nil="true"/>
<RepeatsFriday i:nil="true"/>
<RepeatsSaturday i:nil="true"/>
<DueUnit i:nil="true"/>
<DueInterval i:nil="true"/>
<PrepopulatePriorAnswers>false</PrepopulatePriorAnswers>
<EmailSubject>Vendor Internal Assessment</EmailSubject>
<EmailBody>Vendor Internal Assessment</EmailBody>
<SendReviewerOrIssuerEmail i:nil="true"/>
<SendCategoryEmail i:nil="true"/>
<AdministrativeEmailSubject i:nil="true"/>
<AdministrativeEmailBody i:nil="true"/>
<ScheduledAssessmentId>171</ScheduledAssessmentId>
</AssessmentIssuanceItem>

```

XML REQUEST - Vendor Recurring Assessment (Results.xml)

```

<IssueAssessment>
  <assessmentIssuance xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns:a="http://www.w3.org/2001/XMLSchema">
    <TableId>10066</TableId>
    <FieldId>1439</FieldId>
    <ContentId>6</ContentId>
    <VendorId>6</VendorId>
    <Name>Vendor Recurring Assessment [Vendor Name]</Name>
    <TemplateId>21</TemplateId>
    <VendorContactId>1059</VendorContactId>
    <AllowDelegation>false</AllowDelegation>
    <GenerateFindings>false</GenerateFindings>
  </assessmentIssuance>
</IssueAssessment>

```

```

    <AssignedUserOnly>false</AssignedUserOnly>

    <ShowUserScores>false</ShowUserScores>

    <IssuanceScheduleType>recurring</IssuanceScheduleType>

    <BeginningDate>03/08/2021</BeginningDate>

    <RepeatUnit>Monthly</RepeatUnit>

    <EndingDate>11/05/2022</EndingDate>

    <RepeatInterval>1</RepeatInterval>

    <DueUnit>Weeks</DueUnit>

    <DueInterval>1</DueInterval>

    <EmailSubject>Vendor Recurring Assessment</EmailSubject>

    <EmailBody>Vendor Recurring Assessment [Vendor Name]</EmailBody>

    <SendReviewerOrIssuerEmail>false</SendReviewerOrIssuerEmail>

    </assessmentIssuance>

</IssueAssessment>

```

XML RESPONSE - Vendor Recurring Assessment

```

<AssessmentIssuanceItem xmlns:i="http://www.w3.org/2001/XMLSchema-instance">
    <ProjectId i:nil="true"/>
    <TableId>10066</TableId>
    <FieldId>1439</FieldId>
    <ContentId>6</ContentId>
    <VendorId>6</VendorId>
    <IsVendorInternalMode i:nil="true"/>
    <Name>Vendor Recurring Assessment [Vendor Name]</Name>
    <TemplateId>21</TemplateId>
    <UserIds i:nil="true"/>
    <VendorContactId>1059</VendorContactId>
    <GroupIds i:nil="true"/>
    <AllowDelegation>false</AllowDelegation>
    <AssignedUserOnly i:nil="true"/>
    <ReviewerId i:nil="true"/>
    <GenerateFindings>false</GenerateFindings>
    <FindingsOutputTableId i:nil="true"/>
    <FindingsOutputFieldId i:nil="true"/>

```

```

<FindingsParentGroupingFieldId i:nil="true"/>
<FindingsControlsFieldId i:nil="true"/>
<ShowUserScores>false</ShowUserScores>
<IssuanceScheduleType>recurring</IssuanceScheduleType>
<BeginningDate>03/08/2021</BeginningDate>
<IssueDate i:nil="true"/>
<DueDate i:nil="true"/>
<EndingDate i:nil="true"/>
<RepeatUnit>Monthly</RepeatUnit>
<RepeatInterval>1</RepeatInterval>
<RepeatsSunday i:nil="true"/>
<RepeatsMonday i:nil="true"/>
<RepeatsTuesday i:nil="true"/>
<RepeatsWednesday i:nil="true"/>
<RepeatsThursday i:nil="true"/>
<RepeatsFriday i:nil="true"/>
<RepeatsSaturday i:nil="true"/>
<DueUnit>Weeks</DueUnit>
<DueInterval>1</DueInterval>
<PrepopulatePriorAnswers i:nil="true"/>
<EmailSubject>Vendor Recurring Assessment</EmailSubject>
<EmailBody>Vendor Recurring Assessment [Vendor Name]</EmailBody>
<SendReviewerOrIssuerEmail>false</SendReviewerOrIssuerEmail>
<SendCategoryEmail i:nil="true"/>
<AdministrativeEmailSubject i:nil="true"/>
<AdministrativeEmailBody i:nil="true"/>
<ScheduledAssessmentId>173</ScheduledAssessmentId>
</AssessmentIssuanceItem>

```

JSON Examples

JSON REQUEST (cURL)

```
curl -b cookie.txt -H "content-type: application/json" -H "Accept: application/json" -X POST -d @Results.json
```

```
"https://api.lockpath.app:4443/AssessmentService/IssueAssessment"
```

JSON REQUEST - Immediate Assessment (Results.json)

```
{
  "IssueAssessment": {
    "assessmentIssuance": {
      "ProjectId": 16,
      "Name": "Assessment Issues Today",
      "TemplateId": 21,
      "UserIds": 1027,
      "GroupIds": "",
      "AllowDelegation": false,
      "GenerateFindings": false,
      "AssignedUserOnly": false,
      "ShowUserScores": false,
      "IssuanceScheduleType": "immediate",
      "DueDate": "3/12/2021",
      "PrepopulatePriorAnswers": false,
      "EmailSubject": "Immediate Assessment",
      "EmailBody": "Immediate Assessment"
    }
  }
}
```

JSON RESPONSE - Immediate Assessment

```
{
  "AssessmentIssuanceItem": {
    "ProjectId": 16,
    "TableId": "",
    "FieldId": "",
  }
}
```

```
"ContentId": "",
"VendorId": "",
"IsVendorInternalMode": "",
"Name": "Assessment Issued Today",
"TemplateId": 21,
"UserIds": 1027,
"VendorContactId": "",
"GroupIds": "",
"AllowDelegation": false,
"AssignedUserOnly": "",
"ReviewerId": "",
"GenerateFindings": false,
"FindingsOutputTableId": "",
"FindingsOutputFieldId": "",
"FindingsParentGroupingFieldId": "",
"FindingsControlsFieldId": "",
"ShowUserScores": false,
"IssuanceScheduleType": "immediate",
"BeginningDate": "",
"IssueDate": "",
"DueDate": "3/12/2021",
"EndingDate": "",
"RepeatUnit": "",
"RepeatInterval": "",
"RepeatsSunday": "",
"RepeatsMonday": "",
"RepeatsTuesday": "",
"RepeatsWednesday": "",
"RepeatsThursday": "",
"RepeatsFriday": "",
"RepeatsSaturday": "",
"DueUnit": "",
```

```

        "DueInterval": "",
        "PrepopulatePriorAnswers": false,
        "EmailSubject": "Immediate Assessment",
        "EmailBody": "Immediate Assessment",
        "SendReviewerOrIssuerEmail": "",
        "SendCategoryEmail": "",
        "AdministrativeEmailSubject": "",
        "AdministrativeEmailBody": "",
        "ScheduledAssessmentId": 165
    }
}

```

JSON REQUEST - Onetime Scheduled Assessment (Results.json)

```

{
    "IssueAssessment": {
        "assessmentIssuance": {
            "ProjectId": 16,
            "Name": "Onetime Scheduled Assessment",
            "TemplateId": 21,
            "UserIds": 1027,
            "GroupIds": "",
            "AllowDelegation": false,
            "GenerateFindings": false,
            "AssignedUserOnly": false,
            "ShowUserScores": false,
            "IssuanceScheduleType": "onetime",
            "IssueDate": "03/05/2021",
            "DueDate": "03/31/2021",
            "PrepopulatePriorAnswers": false,
            "EmailSubject": "Onetime Scheduled Assessment",
            "EmailBody": "Onetime Scheduled Assessment"
        }
    }
}

```

JSON RESPONSE - Onetime Scheduled Assessment

```
{
  "AssessmentIssuanceItem": {
    "ProjectId": 16,
    "TableId": "",
    "FieldId": "",
    "ContentId": "",
    "VendorId": "",
    "IsVendorInternalMode": "",
    "Name": "Onetime Scheduled Assessment",
    "TemplateId": 21,
    "UserIds": 1027,
    "VendorContactId": "",
    "GroupIds": "",
    "AllowDelegation": false,
    "AssignedUserOnly": "",
    "ReviewerId": "",
    "GenerateFindings": false,
    "FindingsOutputTableId": "",
    "FindingsOutputFieldId": "",
    "FindingsParentGroupingFieldId": "",
    "FindingsControlsFieldId": "",
    "ShowUserScores": false,
    "IssuanceScheduleType": "onetime",
    "BeginningDate": "",
    "IssueDate": "03/05/2021",
    "DueDate": "03/31/2021",
    "EndingDate": "",
    "RepeatUnit": "",
    "RepeatInterval": "",
    "RepeatsSunday": "",
    "RepeatsMonday": ""
  }
}
```

```

    "RepeatsTuesday": "",
    "RepeatsWednesday": "",
    "RepeatsThursday": "",
    "RepeatsFriday": "",
    "RepeatsSaturday": "",
    "DueUnit": "",
    "DueInterval": "",
    "PrepopulatePriorAnswers": false,
    "EmailSubject": "Onetime Scheduled Assessment",
    "EmailBody": "Onetime Scheduled Assessment",
    "SendReviewerOrIssuerEmail": "",
    "SendCategoryEmail": "",
    "AdministrativeEmailSubject": "",
    "AdministrativeEmailBody": "",
    "ScheduledAssessmentId": 168
  }
}

```

JSON REQUEST - Recurring Scheduled Assessment (Results.json)

```

{
  "IssueAssessment": {
    "assessmentIssuance": {
      "TableId": 10383,
      "FieldId": 6543,
      "ContentId": 1,
      "Name": "Recurring Scheduled Assessment",
      "TemplateId": 21,
      "UserIds": 1027,
      "GroupIds": "",
      "AllowDelegation": false,
      "GenerateFindings": false,
      "AssignedUserOnly": false,
      "ShowUserScores": false,
      "IssuanceScheduleType": "recurring",

```

```

    "BeginningDate": "03/03/2021",
    "RepeatUnit": "monthly",
    "EndingDate": "11/05/2021",
    "RepeatInterval": 1,
    "DueUnit": "daily",
    "DueInterval": 1,
    "EmailSubject": "Recurring Scheduled Assessment",
    "EmailBody": "Recurring Scheduled Assessment"
  }
}

```

JSON RESPONSE - Recurring Scheduled Assessment

```

{
  "AssessmentIssuanceItem": {
    "ProjectId": "",
    "TableId": 10383,
    "FieldId": 6543,
    "ContentId": 1,
    "VendorId": "",
    "IsVendorInternalMode": "",
    "Name": "Recurring Scheduled Assessment",
    "TemplateId": 21,
    "UserIds": 1027,
    "VendorContactId": "",
    "GroupIds": "",
    "AllowDelegation": false,
    "AssignedUserOnly": "",
    "ReviewerId": "",
    "GenerateFindings": false,
    "FindingsOutputTableId": "",
    "FindingsOutputFieldId": "",
    "FindingsParentGroupingFieldId": "",
    "FindingsControlsFieldId": ""
  }
}

```

```

        "ShowUserScores": false,
        "IssuanceScheduleType": "recurring",
        "BeginningDate": "03/03/2021",
        "IssueDate": "",
        "DueDate": "",
        "EndingDate": "",
        "RepeatUnit": "monthly",
        "RepeatInterval": 1,
        "RepeatsSunday": "",
        "RepeatsMonday": "",
        "RepeatsTuesday": "",
        "RepeatsWednesday": "",
        "RepeatsThursday": "",
        "RepeatsFriday": "",
        "RepeatsSaturday": "",
        "DueUnit": "daily",
        "DueInterval": 1,
        "PrepopulatePriorAnswers": "",
        "EmailSubject": "Recurring Scheduled Assessment",
        "EmailBody": "Recurring Scheduled Assessment",
        "SendReviewerOrIssuerEmail": "",
        "SendCategoryEmail": "",
        "AdministrativeEmailSubject": "",
        "AdministrativeEmailBody": "",
        "ScheduledAssessmentId": 169
    }
}

```

JSON REQUEST - Vendor Internal Assessment (Results.json)

```

{
    "IssueAssessment": {
        "assessmentIssuance": {
            "TableId": 10066,
            "FieldId": 1439,

```

```

        "ContentId": 7,
        "VendorId": 7,
        "Name": "Vendor Internal Assessment",
        "TemplateId": 21,
        "VendorContactId": 1048,
        "AllowDelegation": false,
        "GenerateFindings": false,
        "AssignedUserOnly": false,
        "ShowUserScores": false,
        "IssuanceScheduleType": "immediate",
        "DueDate": "03/12/2021",
        "PrepopulatePriorAnswers": false,
        "EmailSubject": "Vendor Internal Assessment",
        "EmailBody": "Vendor Internal Assessment"
    }
}

```

JSON RESPONSE - Vendor Internal Assessment

```

{
    "AssessmentIssuanceItem": {
        "ProjectId": "",
        "TableId": 10066,
        "FieldId": 1439,
        "ContentId": 7,
        "VendorId": 7,
        "IsVendorInternalMode": "",
        "Name": "Vendor Internal Assessment",
        "TemplateId": 21,
        "UserIds": "",
        "VendorContactId": 1048,
        "GroupIds": "",
        "AllowDelegation": false,
        "AssignedUserOnly": ""
    }
}

```

```

    "ReviewerId": "",
    "GenerateFindings": false,
    "FindingsOutputTableId": "",
    "FindingsOutputFieldId": "",
    "FindingsParentGroupingFieldId": "",
    "FindingsControlsFieldId": "",
    "ShowUserScores": false,
    "IssuanceScheduleType": "immediate",
    "BeginningDate": "",
    "IssueDate": "",
    "DueDate": "03/12/2021",
    "EndingDate": "",
    "RepeatUnit": "",
    "RepeatInterval": "",
    "RepeatsSunday": "",
    "RepeatsMonday": "",
    "RepeatsTuesday": "",
    "RepeatsWednesday": "",
    "RepeatsThursday": "",
    "RepeatsFriday": "",
    "RepeatsSaturday": "",
    "DueUnit": "",
    "DueInterval": "",
    "PrepopulatePriorAnswers": false,
    "EmailSubject": "Vendor Internal Assessment",
    "EmailBody": "Vendor Internal Assessment",
    "SendReviewerOrIssuerEmail": "",
    "SendCategoryEmail": "",
    "AdministrativeEmailSubject": "",
    "AdministrativeEmailBody": "",
    "ScheduledAssessmentId": 171
  }

```

```
}
```

JSON REQUEST - Vendor Recurring Assessment (Results.json)

```
{
  "IssueAssessment": {
    "assessmentIssuance": {
      "TableId": 10066,
      "FieldId": 1439,
      "ContentId": 6,
      "VendorId": 6,
      "Name": "Vendor Recurring Assessment [Vendor Name]",
      "TemplateId": 21,
      "VendorContactId": 1059,
      "AllowDelegation": false,
      "GenerateFindings": false,
      "AssignedUserOnly": false,
      "ShowUserScores": false,
      "IssuanceScheduleType": "recurring",
      "BeginningDate": "03/08/2021",
      "RepeatUnit": "Monthly",
      "EndingDate": "11/05/2022",
      "RepeatInterval": 1,
      "DueUnit": "Weeks",
      "DueInterval": 1,
      "EmailSubject": "Vendor Recurring Assessment",
      "EmailBody": "Vendor Recurring Assessment [Vendor Name]",
      "SendReviewerOrIssuerEmail": false
    }
  }
}
```

JSON RESPONSE - Vendor Recurring Assessment

```
{
  "AssessmentIssuanceItem": {
```

```
"ProjectId": "",
"TableId": 10066,
"FieldId": 1439,
"ContentId": 6,
"VendorId": 6,
"IsVendorInternalMode": "",
"Name": "Vendor Recurring Assessment [Vendor Name]",
"TemplateId": 21,
"UserIds": "",
"VendorContactId": 1059,
"GroupIds": "",
"AllowDelegation": false,
"AssignedUserOnly": "",
"ReviewerId": "",
"GenerateFindings": false,
"FindingsOutputTableId": "",
"FindingsOutputFieldId": "",
"FindingsParentGroupingFieldId": "",
"FindingsControlsFieldId": "",
"ShowUserScores": false,
"IssuanceScheduleType": "recurring",
"BeginningDate": "03/08/2021",
"IssueDate": "",
"DueDate": "",
"EndingDate": "",
"RepeatUnit": "Monthly",
"RepeatInterval": 1,
"RepeatsSunday": "",
"RepeatsMonday": "",
"RepeatsTuesday": "",
"RepeatsWednesday": "",
"RepeatsThursday": "",
```

```
"RepeatsFriday": "",
"RepeatsSaturday": "",
"DueUnit": "Weeks",
"DueInterval": 1,
"PrepopulatePriorAnswers": "",
"EmailSubject": "Vendor Recurring Assessment",
"EmailBody": "Vendor Recurring Assessment [Vendor Name]",
"SendReviewerOrIssuerEmail": false,
"SendCategoryEmail": "",
"AdministrativeEmailSubject": "",
"AdministrativeEmailBody": "",
"ScheduledAssessmentId": 173
}
}
```

Appendices

A: Language Identifiers

This appendix includes language IDs that can be utilized in the GetUser, CreateUser, and UpdateUser calls.

Language IDs

The Language object of the GetUser, CreateUser, and UpdateUser calls determine the language in the NAVEX IRM Platform. In addition to English, which is the default language, the NAVEX IRM Platform provides other languages that can be utilized in the language object.

Language names and language IDs

Language Name	Language ID	Language Name	Language ID	Language Name	Language ID
Afrikaans	54	Hindi	57	Polish	21
Albanian	28	Hungarian	14	Portuguese	22
Alsatian	132	Icelandic	15	Punjabi	70
Amharic	94	Igbo	112	Quechua	107
Arabic	1	Indonesian	33	Romanian	24
Armenian	43	Inuktitut	93	Romansh	23
Assamese	77	Irish	60	Russian	25
Azerbaijani	44	isiXhosa	52	Sakha	133
Bangla	69	isiZulu	53	Sami (Northern)	59
Bashkir	109	Italian	16	Sanskrit	79
Basque	45	Japanese	17	Scottish Gaelic	145
Belarusian	35	Kannada	75	Serbian	31770
Bosnian	30746	Kazakh	63	Sesotho sa Leboa	108
Breton	126	Khmer	83	Setswana	50
Bulgarian	2	K'iche	134	Sinhala	91
Catalan	3	Kinyarwanda	135	Slovak	27
Chinese	30724	Kiswahili	65	Slovenian	36
Corsican	131	Konkani	87	Spanish	10
Croatian	26	Korean	18	Swedish	29
Czech	5	Kyrgyz	64	Syriac	90
Danish	6	Lao	84	Tajik	40
Dari	140	Latvian	38	Tamazight	95
Divehi	101	Lithuanian	39	Tamil	73
Dutch	19	Luxembourgish	110	Tatar	68
English	9	Macedonian (FYROM)	47	Telugu	74

Language Name	Language ID	Language Name	Language ID	Language Name	Language ID
Estonian	37	Malay	62	Thai	30
Faroese	56	Malayalam	76	Tibetan	81
Filipino	100	Maltese	58	Turkish	31
Finnish	11	Maori	129	Turkmen	66
French	12	Mapudungun	122	Ukrainian	34
Frisian	98	Marathi	78	Upper Sorbian	46
Galician	86	Mohawk	124	Urdu	32
Georgian	55	Mongolian	80	Uyghur	128
German	7	Nepali	97	Uzbek	67
Greek	8	Norwegian	20	Vietnamese	42
Greenlandic	111	Occitan	130	Welsh	82
Gujarati	71	Odia	72	Wolof	136
Hausa	104	Pashto	99	Yi	120
Hebrew	13	Persian	41	Yoruba	106

B: Troubleshooting Tips

This appendix provides troubleshooting tips and frequently asked questions for issues that may occur when using the NAVEX IRM API application.

Troubleshooting	188
Frequently Asked Questions	189

Troubleshooting

The following are problems you may encounter using the NAVEX IRM API and a suggested solution for each.

Problem	What to do
Enabling the API	NAVEX IRM API is available only with NAVEX IRM Enterprise Edition or with an additional license in the NAVEX IRM Standard Edition.
Licensing the API	Confirm the NAVEX IRM API is licensed. Go to Setup > Platform > System > Subscription / License Details. NAVEX IRM Data API should be listed in the Connectors field.
User access permissions	Confirm the user has appropriate rights to the component (table), workflow stage for the given record, the record itself (restrict record access), and the specified fields.
User not making API calls	Confirm that the API access option is enabled in the user profile. Go to Setup > Platform > Security > Users. In the Email column, click the name of the user in question, and confirm the API Access field is set to Yes.
Server encountered error processing request. The incoming message has an unexpected message format 'Raw'. Expected formats are xml or json	Verify the message format is correct.
Using the right method (GET vs POST vs DELETE)	Consult this guide for the method appropriate to your request.
Capitalization	API methods are case-sensitive. Verify the case for the given method in this guide.
Having set-cookie not cookie	Verify the correct cookie file is specified from the login.
A valid session is required for API request	If you previously had a valid cookie, the session has now expired. Consider creating a custom security configuration with an extended timeout and assign the new configuration to your API user(s) if the session is expiring inappropriately.
Input string is not in the correct format	Review the syntax of your command.
cURL command results in error	API methods are case-sensitive. Verify that you are entering the proper syntax and case for the cURL command as documented in this guide.
Order of switches for cURL command	For a list of curl commands and options, type curl -help at a command prompt.

Frequently Asked Questions

The following are frequently asked questions about the NAVEX IRM API.

Question	Answer
Why should I choose to use NAVEX IRM API?	The NAVEX IRM API is extremely useful for building automated linkages to your internal systems for performing bi-directional updates. You can also use the REST API methods to update/import one-to-many lookup fields for fully managing a record.
In what editions are the NAVEX IRM API available?	NAVEX IRM Enterprise Edition and with an additional license in the NAVEX IRM Standard Edition.
Can I modify user information using the NAVEX IRM API?	Yes, you can add, update, and assign security roles to users and groups using API calls.
How are API licenses counted?	Users with Portal access may have access to the NAVEX IRM API without using an additional user license. If a user account is established specifically for API access, there must be a full user license available for granting the appropriate access.
What methods are supported?	The REST API supports the following HTTP verbs: GET, POST, and DELETE
Is there a limit to the number of calls I can make?	The API is limited to 10 active sessions at one time and up to 20 requests can be processed per second. When retrieving multiple (bulk) records, up to 1,000 records can be requested at once. Messaging is provided if a request is unable to be completed to allow for subsequent attempts.
Will NAVEX program against the NAVEX IRM API on our behalf?	You can engage your NAVEX representative for a statement of work. They will contact you to gather the project requirements and to estimate the expected deliverables to best fit your needs.